

Understanding Masking in NumPy

using the QG Jet Dataset

Boolean Masking vs Masked Arrays



1D Array

1	2	3
---	---	---

```
array([1, 2, 3])
```

2D Array

1	2	3
1	2	3
1	2	3

```
array([[1, 2, 3],  
       [1, 2, 3],  
       [1, 2, 3]])
```

3D Array

		1	2	3
		1	2	3
1	2	3	3	3
1	2	3	3	3
1	2	3	3	3

```
array([[[1, 2, 3],  
        [1, 2, 3],  
        [1, 2, 3]],  
       [[1, 2, 3],  
        [1, 2, 3],  
        [1, 2, 3]],  
       [[1, 2, 3],  
        [1, 2, 3],  
        [1, 2, 3]]])
```

Author:

Pratap Yadav

Institution:

KIIT University

GitHub Repository:

<https://github.com/7hepratapyadav/numpy-masking-qg-jets>

Contents

1	Introduction	9
1.1	Project Objectives	9
2	Dataset Background	10
2.1	What is a Quark?	10
2.2	What is a Gluon?	10
2.3	What is a Jet?	11
2.4	Particle-Level Features	11
2.5	Why Padding is Necessary	11
3	Understanding the Dataset Structure	13
3.1	Loading the Dataset	13
3.2	Inspecting Dataset Keys	14
3.3	Extracting the Feature and Label Arrays	14
3.4	Understanding the Dataset Dimensions	15
3.5	Interpreting <code>X.shape</code>	15
3.6	Meaning of the Four Features	16
3.7	Interpreting <code>y.shape</code>	16
3.8	Inspecting the First Jet	17
3.9	First Observation: Not Every Row Contains Data	17
3.10	Why This Matters	18
4	Understanding Padding	19
4.1	Why Padding Exists	19
4.2	Visual Representation of Padding	20
4.3	Analyzing the First Jet	21
4.4	Counting Padding Rows	21
4.5	Interpreting the Result	24
4.6	Counting Real Particles	24
4.7	Consistency Check	26
4.8	Why This Matters	26

5	Visualizing Particle Presence	28
5.1	Creating a Particle Presence Mask	28
5.2	Visualizing the Mask	31
5.3	Interpreting the Visualization	31
6	Introduction to Boolean Masking	32
6.1	A Simple Example	33
6.2	Applying the Same Idea to Jets	33
7	Constructing a Particle Mask	34
7.1	Step 1: Creating a Boolean Comparison Matrix	34
7.2	Step 2: Reducing Feature-Level Information to Particle-Level Information	36
7.3	Step 3: Constructing the Particle Mask	41
7.4	Step 4: Combining the Operations	42
7.5	What Happens Next?	43
8	Applying the Boolean Mask	44
8.1	Filtering the Jet	44
8.2	Inspecting the Result	45
8.3	Understanding the Shape Transformation	45
8.4	Visual Interpretation	46
9	Understanding Boolean Indexing	47
9.1	How NumPy Interprets the Mask	48
9.2	Internal Selection Process	48
9.3	Why the Shapes Match	49
9.4	Inspecting the Filtered Jet	49
10	Physics Interpretation	49
10.1	Why Physicists Remove Padding	50
10.2	Applications of Boolean Masking	50
10.3	Limitation of Boolean Masking	51
11	NumPy Masked Arrays (numpy.ma)	52
12	Creating a Masked Array	53

12.1 Step 1: Defining the Masking Condition	53
12.2 Step 2: Passing the Original Data	54
12.3 What <code>masked_where()</code> Actually Does	54
13 Internal Structure of a Masked Array	55
13.1 Data Array	55
13.2 Mask Array	55
13.3 Meaning of the Mask Values	55
14 Inspecting the Masked Array	56
14.1 Why Does NumPy Display <code>--</code> ?	56
15 First Comparison with Boolean Masking	57
16 Understanding the Shape of a Masked Array	59
16.1 Why the Shape Does Not Change	59
16.2 Comparing the Two Approaches	60
17 Why Preserving Shape Is Useful	61
18 Automatic Exclusion of Masked Values	62
18.1 Why This Matters	63
19 Visualizing the Padding Mask	63
19.1 Interpreting the Heatmap	65
19.2 The Transition Region	65
19.3 Why This Visualization Is Useful	66
20 Preparing for Quantitative Comparison	67
21 Comparing Particle Counts Using Both Masking Techniques	68
21.1 Particle Counting Using Boolean Masking	68
21.2 Particle Counting Using Masked Arrays	69
21.3 Understanding the Calculation	70
21.4 Comparing the Results	72
22 Comparing the Mean Transverse Momentum	72

22.1 Mean p_T Without Masking	73
22.2 Mean p_T Using Boolean Masking	73
22.3 Mean p_T Using Masked Arrays	74
23 Why Does the Mean Change So Dramatically?	74
24 Physical Interpretation	75
25 Summary of Results	76
26 Extending Boolean Masking to the Entire Dataset	77
27 Constructing a Dataset-Level Particle Mask	77
27.1 Step 1: Comparing the Entire Dataset with Zero	78
27.2 Step 2: Reducing Feature-Level Information	79
27.3 Interpreting the Dataset-Level Mask	79
27.4 Step 3: Counting Particles in Each Jet	80
28 Understanding the Output Shape	80
29 Visualizing the Particle Count Distribution	81
30 Interpreting the Histogram	82
31 Physics Interpretation of Particle Multiplicity	83
32 What the Histogram Reveals About Padding	84
33 Why This Analysis Matters	85
34 Preparing for Jet-Level Filtering	85
35 Dataset-Level Masking vs Jet-Level Masking	86
36 Dataset-Level Masking	86
36.1 When Dataset-Level Masking Is Useful	87
37 Jet-Level Masking	87
37.1 Breaking Down the Expression	88

38 Inspecting the Filtered Jets	89
38.1 Why the Shapes Differ	89
39 Variable-Length Outputs	90
39.1 Why a Python List Is Required	91
40 Why We Cannot Simply Write	91
40.1 What Actually Happens	92
40.2 Information Is Lost	92
41 Dataset-Level vs Jet-Level Masking	93
42 Key Takeaway	93
43 Applying Masked Arrays to the Entire Dataset	95
44 Creating a Dataset-Wide Masked Array	95
44.1 What Does <code>X_masked</code> Contain?	96
45 Computing Particle Counts for Every Jet	96
45.1 Step 1: Accessing the Mask Tensor	97
45.2 Step 2: Determining Whether a Particle Is Fully Masked	97
45.3 Step 3: Inverting the Logic	98
45.4 Step 4: Counting Particles	99
46 Inspecting the Results	99
47 Computing the Mean Transverse Momentum of Every Jet	100
47.1 Understanding the Slice	100
47.2 Applying the Mean	101
48 Inspecting the Mean p_T Results	101
49 Why Masked Arrays Are Powerful for Dataset-Wide Analysis	102
49.1 Automatic Statistical Handling	103
50 Dataset-Wide Comparison	103

51 Preparing for the Final Comparison	104
52 Boolean Masking vs Masked Arrays	105
53 A Common Objective	105
54 Core Philosophy of Each Approach	106
54.1 Boolean Masking	106
54.2 Masked Arrays	106
55 Structural Differences	107
55.1 Boolean Masking	107
55.2 Masked Arrays	107
56 Effect on Memory Usage	108
57 Effect on Statistical Calculations	108
57.1 Boolean Masking	108
57.2 Masked Arrays	109
58 Machine Learning Perspective	109
59 Particle Physics Perspective	109
60 Advantages of Boolean Masking	110
61 Advantages of Masked Arrays	110
62 Limitations of Boolean Masking	111
63 Limitations of Masked Arrays	111
64 Comprehensive Comparison	112
65 Practical Guidelines	112
66 Final Observation	113
67 Key Takeaways	114
67.1 Real-World Scientific Datasets Often Contain Padding	114

67.2 Padding Can Distort Statistical Calculations	114
67.3 Boolean Masking Converts Physical Questions into Logical Operations .	115
67.4 Boolean Masking Physically Removes Padding	116
67.5 Masked Arrays Preserve the Original Tensor Structure	116
67.6 Different Methods Can Produce the Same Physical Result	117
67.7 Dataset-Level and Jet-Level Analysis Are Fundamentally Different	118
67.8 NumPy Enables Large-Scale Vectorized Processing	118
67.9 Masking Is More Than a NumPy Feature	119
67.10 Choosing the Right Masking Strategy Depends on the Problem	119
68 Summary of the Learning Journey	120
69 Conclusion	121
70 What Was Learned	121
71 Understanding Boolean Masking	122
72 Understanding Masked Arrays	122
73 Comparing the Two Approaches	123
74 The Importance of Handling Padding Correctly	124
75 Broader Relevance Beyond This Dataset	124
76 Personal Reflection on the Exploration	125
77 Final Remarks	126
78 References	127

Abstract

In many scientific datasets, observations naturally contain variable amounts of information. Machine learning models, however, require fixed-size numerical inputs. To satisfy this requirement, datasets are often padded with artificial values. While padding simplifies storage and computation, it introduces entries that do not represent real physical information.

The Quark–Gluon (QG) Jet Dataset used in this project presents exactly this challenge. Different jets contain different numbers of particles, yet every jet must be stored using the same tensor dimensions. Consequently, unused particle positions are filled with zeros.

This report explores two masking techniques available in NumPy:

- Boolean Masking
- Masked Arrays (`numpy.ma`)

Using a real particle physics dataset, we investigate how padding affects statistical analysis, how masking techniques identify valid particles, and how different masking approaches influence the structure of the resulting data.

The goal of this project is not only to implement masking techniques but also to develop a conceptual understanding of why masking is necessary and how it is used in scientific computing and machine learning.

1 Introduction

Modern machine learning systems require data to be represented in structured numerical formats. In many applications, every observation contains the same number of features, making storage straightforward.

Scientific datasets, however, often violate this assumption.

In particle physics, different collision events naturally produce different numbers of particles. As a result, observations can have variable lengths. Since NumPy arrays and machine learning models generally expect fixed dimensions, this variability introduces a significant challenge.

To solve this problem, physicists commonly use a technique known as **padding**. Padding extends shorter observations by adding artificial entries until every observation reaches a common size.

Although padding allows data to be stored efficiently, it introduces values that do not correspond to real physical particles. These artificial entries must be handled carefully because they can distort statistical calculations and machine learning features.

Masking provides a solution to this problem.

Masking enables us to distinguish genuine particle information from artificial padding and ensures that calculations are performed only on physically meaningful data.

In this project, we explore masking techniques using NumPy and apply them to a real Quark–Gluon jet dataset.

1.1 Project Objectives

The primary objectives of this project are:

1. Understand why padding exists in particle physics datasets.
2. Learn how Boolean Masking works.
3. Learn how NumPy Masked Arrays work.
4. Compare both techniques conceptually and practically.
5. Investigate how padding affects statistical calculations.
6. Apply masking to both individual jets and the entire dataset.
7. Identify practical use cases for each masking approach.

By the end of this report, the reader should have a complete understanding of how masking techniques are used to process padded scientific datasets.

2 Dataset Background

The dataset used throughout this project is:

`QG_jets.npz`

where:

$Q \rightarrow$ Quark

$G \rightarrow$ Gluon

To understand the significance of this dataset, it is useful to review several fundamental concepts from particle physics.

2.1 What is a Quark?

Quarks are elementary particles that serve as the building blocks of hadrons such as protons and neutrons.

They possess several important physical properties:

- Electric charge
- Color charge
- Spin
- Mass

Quarks participate in the strong nuclear force and cannot exist in isolation due to a phenomenon known as confinement.

2.2 What is a Gluon?

Gluons are the force carriers of the strong interaction.

Just as photons mediate the electromagnetic force, gluons mediate the strong force between quarks.

Unlike photons, gluons themselves carry color charge, allowing them to interact with one another and generate complex particle dynamics.

2.3 What is a Jet?

When particles collide at extremely high energies, quarks and gluons may be produced.

These particles cannot remain isolated and rapidly undergo a process known as hadronization. During this process, they generate cascades of secondary particles.

The resulting collection of particles appears experimentally as a narrow spray called a **jet**.

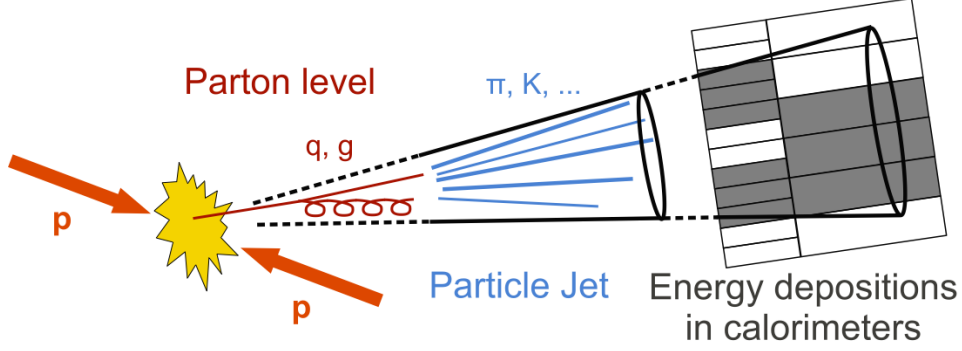


Figure 1: Simplified representation of jet formation.

2.4 Particle-Level Features

Each particle in the dataset is represented using four features:

Feature	Description
p_T	Transverse Momentum
y	Rapidity
ϕ	Azimuthal Angle
pid	Particle Identifier

Table 1: Particle-level features stored in the QG dataset.

2.5 Why Padding is Necessary

Different jets contain different numbers of particles.

For example:

Jet A	18 particles
Jet B	35 particles
Jet C	72 particles
Jet D	139 particles

Machine learning models require fixed-size inputs.

Therefore, every jet is stored using a common maximum capacity of 139 particle positions.

Unused particle positions are filled with zeros.

```
[ 0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00]
[ 0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00]
[ 0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00]
[ 0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00]
[ 0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00]
[ 0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00]
[ 0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00]
[ 0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00]
[ 0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00]
[ 0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00]
[ 0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00]
[ 0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00]
[ 0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00]
```

Figure 2: Illustration of padding within a jet.

These padded rows do not represent physical particles and must be excluded from analysis.

This requirement motivates the masking techniques explored throughout the remainder of this report.

3 Understanding the Dataset Structure

Before applying any masking technique, it is essential to understand how the dataset is organized.

Masking operations depend heavily on the structure of the underlying data. Without understanding the dimensions of the dataset and the meaning of each axis, it becomes difficult to interpret the results of masking operations correctly.

In this section, we will:

- Load the dataset.
- Inspect its contents.
- Understand the meaning of each dimension.
- Examine an individual jet.
- Identify the presence of padding.

3.1 Loading the Dataset

The first step is loading the compressed NumPy dataset.

```
data = np.load('QG_jets.npz')
```

Purpose

The file `QG_jets.npz` contains the complete dataset used throughout this project.

NumPy provides the function:

```
np.load()
```

which is capable of reading compressed NumPy files and returning their contents.

The object returned by `np.load()` behaves similarly to a Python dictionary. Instead of directly returning arrays, it provides access to arrays through named keys.

Conceptually:

`QG_jets.npz`

contains multiple arrays packaged into a single file.

The variable:

```
data
```

acts as a container that stores all arrays contained inside the dataset.

3.2 Inspecting Dataset Keys

After loading the dataset, we inspect its available keys.

```
print(data.files)
```

Output:

```
['X', 'y']
```

Interpretation

The output indicates that the dataset contains two arrays:

- X
- y

This structure follows a common convention used in machine learning:

- X contains input features.
- y contains labels.

Therefore:

X	Jet particle information
y	Jet classification labels

3.3 Extracting the Feature and Label Arrays

The arrays can now be extracted.

```
X = data['X']  
y = data['y']
```

What Happens Here?

The operation:

```
data['X']
```

retrieves the feature tensor from the dataset.

Similarly:

```
data['y']
```

retrieves the label array.

After these assignments:

```
X
```

contains all particle-level information,

while:

```
y
```

contains the target labels used for classification.

3.4 Understanding the Dataset Dimensions

To understand the structure of the dataset, we inspect the shapes of both arrays.

```
print(X.shape)
print(y.shape)
```

Output:

```
(100000, 139, 4)
(100000,)
```

At first glance, these dimensions may appear intimidating.

However, each dimension has a clear physical meaning.

3.5 Interpreting X.shape

The feature tensor has shape:

(100000, 139, 4)

This can be interpreted as:

100000 → Number of Jets
 139 → Maximum Number of Particles per Jet
 4 → Features per Particle

A useful visualization is:

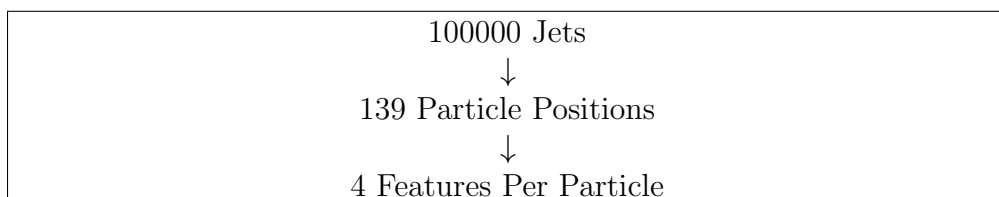


Figure 3: Hierarchical structure of the feature tensor.

This tells us that every jet is stored as a matrix containing:

- Up to 139 particles.
- Four features for every particle.

3.6 Meaning of the Four Features

Each particle is represented by four numerical quantities.

Feature	Physical Meaning
p_T	Transverse Momentum
y	Rapidity
ϕ	Azimuthal Angle
pid	Particle Identifier

Table 2: Particle-level features stored for every constituent.

Consequently, every particle can be represented as:

$$(p_T, y, \phi, pid)$$

For example:

$$(2.68, 0.35, 4.74, 22)$$

describes a single particle using four physical quantities.

3.7 Interpreting `y.shape`

The label array has shape:

$$(100000,)$$

This means:

- One label exists for every jet.
- There are 100000 labels.

Thus:

$$x[i]$$

contains the particle information for Jet i ,
while:

$y[i]$

contains the class label for the same jet.

3.8 Inspecting the First Jet

To better understand the structure of the dataset, we inspect the first jet.

```
print(X[0])
```

The output reveals a two-dimensional matrix.

Each row corresponds to a particle.

Each column corresponds to a feature.

Conceptually:

$$X[0] = \begin{bmatrix} p_T & y & \phi & pid \\ p_T & y & \phi & pid \\ p_T & y & \phi & pid \\ \vdots & \vdots & \vdots & \vdots \end{bmatrix}$$

Therefore:

- Row index $\rightarrow$ particle.
- Column index $\rightarrow$ feature.

3.9 First Observation: Not Every Row Contains Data

A careful inspection of the first jet reveals an important pattern.

The first several rows contain meaningful numerical values:

```
[0.268, 0.356, 4.741, 22]
[0.160, -0.255, 4.550, 22]
...
```

However, further down the matrix we encounter rows of the form:

```
[0.0, 0.0, 0.0, 0.0]
```

These rows do not correspond to real particles.

Instead, they were inserted during the padding process.

This observation immediately suggests that:

- Not all particle positions are occupied.
- Some rows represent genuine particles.
- Some rows represent padding.

The challenge now becomes distinguishing these two categories.

3.10 Why This Matters

Suppose we compute the mean transverse momentum of the jet.

If padded rows are treated as real particles, the calculation will include many artificial zeros.

Consequently:

- Mean values become smaller.
- Statistical distributions become distorted.
- Physical interpretations become incorrect.

Therefore, before performing any meaningful analysis, we must identify which rows correspond to real particles and which rows correspond to padding.

This motivates the next section, where we investigate the padding structure of the dataset and develop a systematic method for detecting padded particles.

4 Understanding Padding

In the previous section, we learned that every jet in the dataset is stored using a fixed tensor shape:

$$(139, 4)$$

where:

- 139 represents the maximum number of particle positions.
- 4 represents the number of features stored for each particle.

However, not every jet actually contains 139 particles.

This raises an important question:

What occupies the unused particle positions?

The answer is **padding**.

Padding is one of the most common techniques used in machine learning and scientific computing whenever observations naturally have variable lengths but must be stored using a fixed-size structure.

Before introducing masking techniques, it is essential to understand how padding appears in the dataset and why it creates problems for statistical analysis.

4.1 Why Padding Exists

Consider the following hypothetical jets:

Jet	Number of Particles
Jet A	18
Jet B	42
Jet C	75
Jet D	139

Each jet contains a different number of particles.

Unfortunately, NumPy arrays require rectangular dimensions.

Therefore, the following structure is impossible:

```
Jet A -> 18 particles
Jet B -> 42 particles
Jet C -> 75 particles
Jet D -> 139 particles
```

because each row would have a different length.

To solve this problem, every jet is expanded to the same maximum size.

In the QG dataset:

$$\text{Maximum Number of Particles} = 139$$

Therefore:

$$\text{Jet A} \rightarrow 18 \text{ real particles} + 121 \text{ padding rows}$$

$$\text{Jet B} \rightarrow 42 \text{ real particles} + 97 \text{ padding rows}$$

$$\text{Jet C} \rightarrow 75 \text{ real particles} + 64 \text{ padding rows}$$

$$\text{Jet D} \rightarrow 139 \text{ real particles} + 0 \text{ padding rows}$$

The padding rows are represented by zeros.

4.2 Visual Representation of Padding

A simplified jet may look like:

$$\begin{bmatrix} 2.5 & 0.1 & 4.7 & 22 \\ 1.8 & -0.2 & 4.5 & 22 \\ 3.2 & 0.3 & 4.6 & -211 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

The first three rows correspond to real particles.

The remaining rows correspond to padding.

Conceptually:

Real Particle
Real Particle
Real Particle
Padding
Padding
Padding

Figure 4: Simplified representation of padded particle positions.

4.3 Analyzing the First Jet

To investigate padding in the dataset, we begin by selecting the first jet.

```
first_jet = X[0]
```

Purpose

The tensor:

$$X$$

contains information for all 100000 jets.

The operation:

```
X[0]
```

extracts only the first jet.

The resulting array has shape:

$$(139, 4)$$

meaning:

- 139 particle positions
- 4 features per particle

This single jet will serve as our example throughout the remainder of the masking discussion.

4.4 Counting Padding Rows

Now that we have isolated the first jet, we would like to determine how many rows correspond to padding.

The following code performs this calculation:

```
padding_rows = np.sum(  
    (first_jet == 0).all(axis=1)  
)
```

At first glance, this expression may appear complicated.

Therefore, let us examine it step by step.

Step 1: Compare Every Value with Zero

The first operation is:

```
first_jet == 0
```

Every element is compared with zero.

A real particle row:

`[2.68, 0.35, 4.74, 22]`

becomes:

`[False, False, False, False]`

because none of its values are zero.

A padded row:

`[0, 0, 0, 0]`

becomes:

`[True, True, True, True]`

because every value is zero.

The resulting array still has shape:

`(139, 4)`

Only the data type changes from numerical values to Boolean values.

Step 2: Check Whether All Features Are Zero

Next:

```
(first_jet == 0).all(axis=1)
```

is applied.

The function:

`all()`

asks:

Are all values in this row True?

Consider:

$[True, True, True, True]$

Since all entries are True:

$all = True$

Now consider:

$[False, False, False, False]$

Since not all entries are True:

$all = False$

The output becomes:

$(139,)$

with one Boolean value per particle.

Step 3: Count the Number of True Values

Finally:

```
np.sum(...)
```

counts the number of True values.

Recall that:

$True = 1$

and

$False = 0$

Therefore:

$np.sum()$

effectively counts the number of padded rows.

The result is:

```
Padding rows: 121
```

4.5 Interpreting the Result

The first jet contains:

121

padding rows.

Since every jet contains 139 particle positions:

$$139 - 121 = 18$$

Therefore, only:

18

positions correspond to genuine particles.

The remaining positions were added solely to maintain a fixed tensor size.

4.6 Counting Real Particles

Instead of counting padding rows, we can directly count real particles.

The following code performs this calculation:

```
real_particles = np.sum(
    (first_jet != 0).any(axis=1)
)
```

Output:

```
Real Particles: 18
```

Again, let us examine the logic carefully.

Step 1: Identify Non-Zero Values

The expression:

```
first_jet != 0
```

creates a Boolean matrix.

For a real particle:

[2.68, 0.35, 4.74, 22]

we obtain:

[*True*, *True*, *True*, *True*]

For a padded row:

[0, 0, 0, 0]

we obtain:

[*False*, *False*, *False*, *False*]

Step 2: Determine Whether a Particle Exists

Next:

```
.any(axis=1)
```

is applied.

The function:

any()

asks:

Does this row contain at least one True value?

For a real particle:

[*True*, *True*, *True*, *True*]

the answer is:

True

For a padded row:

[*False*, *False*, *False*, *False*]

the answer is:

False

Consequently, each row is reduced to a single Boolean value indicating whether the particle exists.

Step 3: Count Existing Particles

The final operation:

```
np.sum(...)
```

counts the number of True values.

The result:

18

represents the number of genuine particles contained in the first jet.

4.7 Consistency Check

Notice that:

$$\text{Real Particles} + \text{Padding Rows} = 139$$

Substituting the measured values:

$$18 + 121 = 139$$

which exactly matches the total number of particle positions available within the jet.

This confirms that our interpretation of the data is correct.

4.8 Why This Matters

Suppose we calculate the mean transverse momentum of the jet without removing padding.

The 121 padded rows contribute artificial zeros.

As a result:

- The mean becomes artificially smaller.
- Physical observables become distorted.
- Statistical analysis becomes unreliable.

- Machine learning features become biased.

Therefore, before performing any meaningful analysis, we must identify which rows correspond to real particles and which correspond to padding.

This requirement naturally leads to the concept of **masking**, which is introduced in the next section.

5 Visualizing Particle Presence

In the previous section, we established that the first jet contains:

18

real particles and

121

padding rows.

Although these numerical values provide useful information, they do not immediately reveal *where* the real particles are located within the jet.

A visual representation can make the structure much easier to understand.

To achieve this, we create a particle presence mask and visualize it using Matplotlib.

5.1 Creating a Particle Presence Mask

The following code constructs a simple numerical representation of particle presence:

```
particle_mask = (
    (first_jet != 0)
    .any(axis=1)
).astype(int)
```

At first glance, this expression may appear complicated.

However, it is simply a sequence of three operations.

1. Compare every value with zero.
2. Determine whether each particle contains at least one non-zero feature.
3. Convert Boolean values into integers.

The result is a one-dimensional array containing only:

0

and

1

values.

Step 1: Identify Non-Zero Values

The expression:

```
first_jet != 0
```

creates a Boolean matrix.

Real particle features become:

True

while padded entries become:

False

The resulting shape remains:

$(139, 4)$

because every particle still contains four features.

Step 2: Determine Whether a Particle Exists

Next:

```
.any(axis=1)
```

is applied.

Recall that:

any()

asks:

Does this row contain at least one True value?

A real particle:

$[True, True, True, True]$

becomes:

True

A padded row:

$$[False, False, False, False]$$

becomes:

$$False$$

The shape is reduced from:

$$(139, 4)$$

to:

$$(139,)$$

because each particle is now represented by a single Boolean value.

Step 3: Convert Boolean Values to Integers

The final operation:

```
.astype(int)
```

converts:

$$True \rightarrow 1$$

and

$$False \rightarrow 0$$

Consequently:

$$particle_mask$$

becomes a binary representation of particle presence.

Conceptually:

$$[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, \dots]$$

where:

- 1 indicates a real particle.
- 0 indicates padding.

5.2 Visualizing the Mask

The mask can now be plotted.

```
plt.figure(figsize=(16,3))  
plt.plot(particle_mask)  
plt.title("Particle Presence Mask")  
plt.xlabel("Particle Index")  
plt.ylabel("Present?")  
plt.show()
```

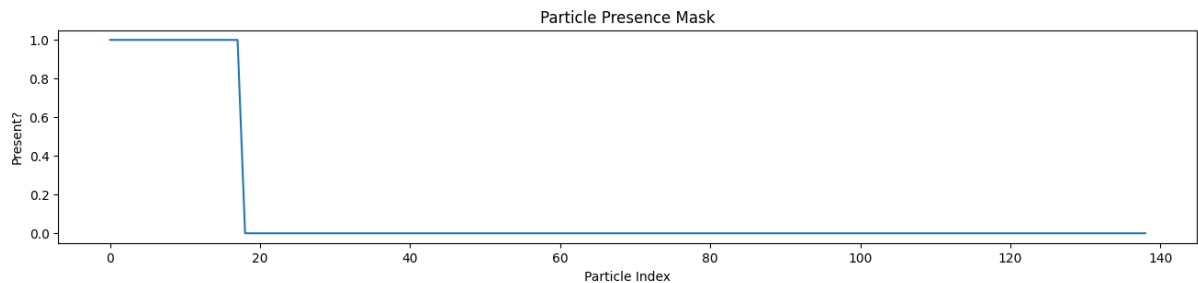


Figure 5: Particle presence mask for the first jet.

5.3 Interpreting the Visualization

The plot provides a direct visual representation of particle occupancy within the jet.

The horizontal axis represents:

Particle Position

while the vertical axis represents:

Particle Presence

A value of:

1

indicates that the particle position contains valid physical information.

A value of:

0

indicates that the position corresponds to padding.

The visualization immediately reveals a clear boundary.

The first several particle positions contain genuine particles, while the remaining positions correspond entirely to padding.

Conceptually:

$$\underbrace{1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1}_{18 \text{ Real Particles}}$$

$$\underbrace{0, 0, 0, 0, 0, 0, 0, \dots}_{121 \text{ Padding Rows}}$$

This confirms the particle counts obtained in the previous section.

6 Introduction to Boolean Masking

Now that we have identified which particle positions contain real data and which positions correspond to padding, we must determine how to remove the padded entries from our analysis.

This leads us to one of the most useful features provided by NumPy:

Boolean Masking

Boolean masking is a data selection technique that uses a Boolean array to determine which elements should be retained and which should be discarded.

Instead of manually looping through the data and checking every row individually, NumPy allows us to perform the selection operation using a mask.

The mask acts as a filter.

Rows associated with:

True

are retained.

Rows associated with:

False

are discarded.

6.1 A Simple Example

Before applying Boolean Masking to particle physics data, let us consider a simple example.

Suppose we have the array:

```
arr = np.array(  
    [5,8,0,12,0,4]  
)
```

and we wish to remove all zeros.

First, we construct a mask:

```
mask = arr != 0
```

The result is:

[True, True, False, True, False, True]

Applying the mask:

```
arr[mask]
```

produces:

[5, 8, 12, 4]

The zeros have been removed automatically.

This is the essence of Boolean Masking.

A Boolean array acts as a selection rule that determines which elements survive in the output.

6.2 Applying the Same Idea to Jets

The exact same principle can be applied to particle physics data.

Instead of filtering individual numbers, we filter entire particle rows.

Our objective is:

- Keep rows containing real particles.
- Remove rows corresponding to padding.

To accomplish this, we must first construct a Boolean mask for the jet.

The construction of this mask is the most important step in the entire masking process.

For this reason, we will examine it carefully and break it into multiple stages.

7 Constructing a Particle Mask

The goal of this section is to create a Boolean mask capable of distinguishing:

- Real particles.
- Padded rows.

The mask will later be used to filter the jet and remove padding.

The complete expression is:

```
mask = (first_jet != 0).any(axis=1)
```

Although compact, this statement performs several operations internally.

To understand it properly, we will decompose it into smaller steps.

The first step is constructing a Boolean comparison matrix.

7.1 Step 1: Creating a Boolean Comparison Matrix

The operation begins with:

```
comparison = first_jet != 0
```

This statement compares every feature value of every particle with zero.

The resulting matrix has exactly the same shape as the original jet:

$$(139, 4)$$

Only the data type changes.

Numerical values become Boolean values.

Real particle row:

$$[2.68, 0.35, 4.74, 22]$$

becomes:

$$[True, True, True, True]$$

Padding row:

$$[0, 0, 0, 0]$$

becomes:

[False, False, False, False]

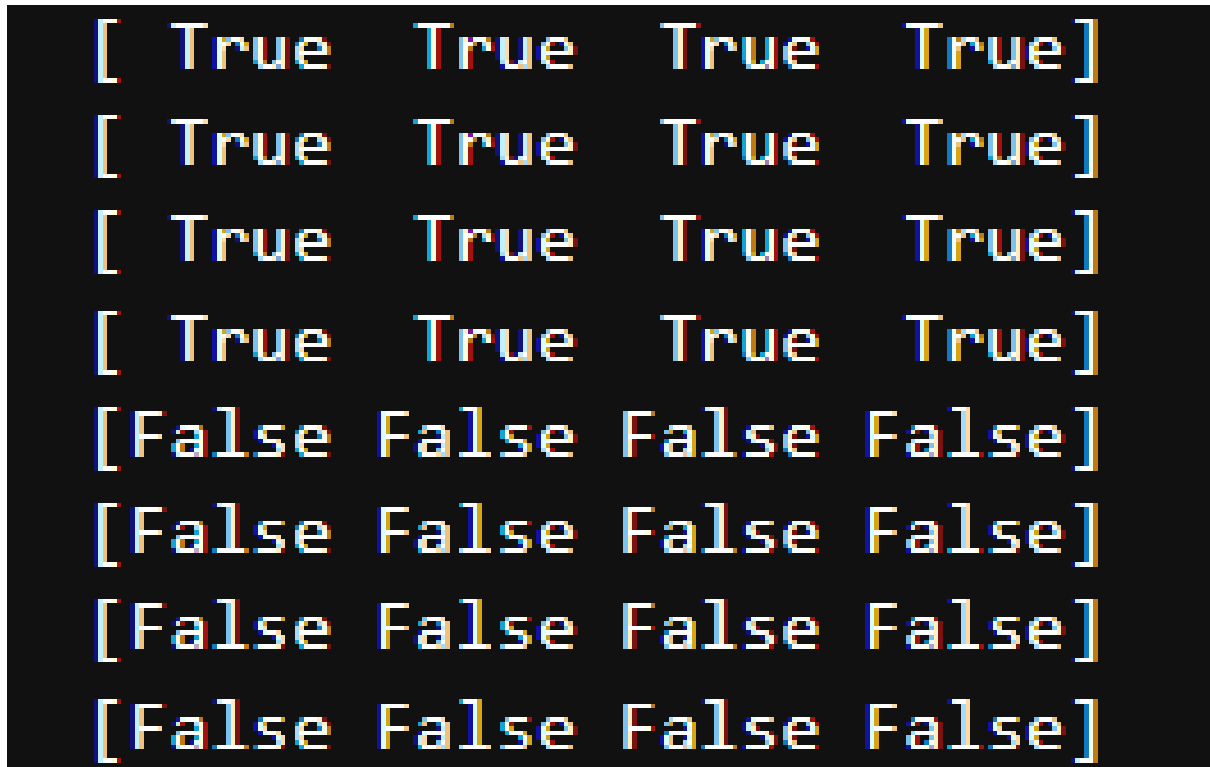


Figure 6: Boolean comparison matrix obtained using `first_jet != 0`.

At this stage, every feature of every particle has been classified as either:

- Valid (True)
- Padded (False)

However, we still have four Boolean values per particle.

The next step is to convert feature-level information into particle-level information.

7.2 Step 2: Reducing Feature-Level Information to Particle-Level Information

In the previous subsection, we created a Boolean comparison matrix:

```
comparison = first_jet != 0
```

The resulting matrix had shape:

$$(139, 4)$$

and contained one Boolean value for every feature of every particle.

Conceptually, the comparison matrix answered the following question:

Is this particular feature value non-zero?

However, our ultimate objective is not to determine whether individual feature values are non-zero.

Instead, we wish to determine:

Does this particle exist?

Therefore, the comparison matrix contains more information than we actually need.

We must now reduce feature-level information into particle-level information.

Why Reduction Is Necessary

Recall that every particle is represented using four features:

$$(p_T, y, \phi, pid)$$

Consider the following particle:

$$[2.68, 0.35, 4.74, 22]$$

After comparison with zero:

$$[True, True, True, True]$$

This tells us that every feature contains meaningful information.

However, for masking purposes, we do not need four separate Boolean values.

We only need one answer:

Does this particle contain any valid information?

If the answer is yes:

True

If the answer is no:

False

Consequently, we must collapse four Boolean values into a single Boolean decision.

Applying `any(axis=1)`

The reduction is performed using:

```
mask = comparison.any(axis=1)
```

The function:

any()

asks:

Does at least one element equal True?

If the answer is yes:

True

is returned.

If every element is False:

False

is returned.

Example 1: Real Particle

Consider:

[True, True, True, True]

Applying:

any()

produces:

True

because at least one element is True.

In fact, all elements are True.

Therefore, the particle is considered valid.

Example 2: Padded Particle

Now consider:

[False, False, False, False]

Applying:

any()

produces:

False

because no feature contains meaningful information.

This row therefore corresponds to padding.

Visual Interpretation

A useful way to visualize the operation is:

[True True True True]

↓

any()

↓

True

Similarly:

[False False False False]

↓

any()

↓

False

The four-dimensional feature representation has now been reduced to a single Boolean decision.

Understanding the Shape Transformation

Before applying:

```
any(axis=1)
```

the comparison matrix had shape:

$(139, 4)$

where:

- 139 = particle positions
- 4 = features per particle

After reduction:

$(139,)$

The feature dimension disappears because the four Boolean values are summarized into a single decision.

Conceptually:

$$(139, 4)$$

$$\downarrow$$

$$\text{any}(\text{axis} = 1)$$

$$\downarrow$$

$$(139,)$$

This transformation is one of the most important concepts in masking.

We have successfully converted:

Feature-Level Information

into

Particle-Level Information

Interpreting the Output

The notebook output is:

```
[ True True True True True True
  True True True True True True
  True True True True True True
  False False False False ...
]
```

The interpretation is straightforward.

The first 18 values are:

True

indicating that the first 18 particle positions contain genuine particles.

The remaining 121 values are:

False

indicating that those positions correspond to padding.

Thus:

18

particle positions are occupied, while

121

positions are empty.

7.3 Step 3: Constructing the Particle Mask

The result produced by:

```
comparison.any(axis=1)
```

is known as a **particle mask**.

The particle mask contains exactly one Boolean value for every particle position within the jet.

Conceptually:

$[True, True, True, True, \dots, False, False]$

Each Boolean value acts as a label describing the status of a particle position.

Mask Value	Interpretation
True	Real Particle
False	Padding

Table 3: Meaning of values in the particle mask.

The mask therefore acts as a compact summary of particle occupancy.

Instead of storing all four particle features, we store only the information required to distinguish real particles from padding.

Why the Particle Mask Is Important

The particle mask serves as the foundation of Boolean Masking.

Without this mask, NumPy has no way to determine:

- Which rows should be retained.
- Which rows should be discarded.

The mask effectively becomes a filtering rule.

Later, when we write:

```
first_jet[mask]
```

NumPy will consult the particle mask to decide which rows survive in the output.

Thus, the particle mask acts as a bridge between:

- Data inspection
- Data filtering

7.4 Step 4: Combining the Operations

Although we have examined the masking process step by step, NumPy allows the entire operation to be written compactly as:

```
mask = (first_jet != 0).any(axis=1)
```

This single statement performs all previous steps internally:

1. Compare every feature value with zero.
2. Create a Boolean comparison matrix.
3. Apply `any(axis=1)`.
4. Construct the particle mask.

The output remains:

```
(139,)
```

and contains:

```
18
```

True values and

```
121
```

False values.

The compact expression is therefore equivalent to the longer step-by-step procedure.

7.5 What Happens Next?

At this point we have successfully identified which particle positions contain genuine physical information.

However, the original jet still contains all 139 rows.

The padded rows have merely been identified—they have not yet been removed.

The next step is to use the particle mask as a filtering rule.

Doing so introduces one of the most powerful features of NumPy:

Boolean Indexing

Using Boolean Indexing, NumPy can automatically select the rows associated with:

True

and discard the rows associated with:

False

This process physically removes padded particles from the dataset and leaves only the genuine jet constituents.

In the next section, we will apply the particle mask to the original jet and examine how Boolean Indexing works internally.

8 Applying the Boolean Mask

In the previous section, we successfully constructed a particle mask:

```
mask = (first_jet != 0).any(axis=1)
```

The mask contains:

139

Boolean values, one for every particle position in the jet.

Conceptually:

[True, True, True, ..., True, False, False, ..., False]

where:

- True indicates a real particle.
- False indicates a padded particle position.

However, the original jet still contains all 139 rows.

At this stage, the padded rows have merely been identified.

They have not yet been removed.

The next step is to use the particle mask to filter the original jet.

8.1 Filtering the Jet

The filtering operation is remarkably simple.

```
jet_filtered = first_jet[mask]
```

Although this statement consists of only a few characters, a significant amount of work is being performed internally by NumPy.

This operation is known as:

Boolean Indexing

Boolean Indexing allows an array of Boolean values to act as a selection rule.

Instead of using numerical indices, NumPy uses the Boolean values to decide which rows should remain in the output.

8.2 Inspecting the Result

The notebook prints:

```
print(first_jet.shape)
print(jet_filtered.shape)
```

Output:

```
(139,4)
(18,4)
```

This output immediately reveals something important.

The shape of the array has changed.

Originally:

first_jet

had shape:

(139,4)

After applying the mask:

jet_filtered

has shape:

(18,4)

This confirms that:

- All padded rows have been removed.
- Only genuine particles remain.
- The number of rows now equals the number of physical particles.

8.3 Understanding the Shape Transformation

Before filtering:

(139,4)

represented:

- 139 particle positions
- 4 features per particle

After filtering:

$(18, 4)$

represents:

- 18 real particles
- 4 features per particle

Notice that the feature dimension remains unchanged.

Only the particle dimension changes.

Conceptually:

$(139, 4)$

↓

Boolean Masking

↓

$(18, 4)$

This is one of the defining characteristics of Boolean Masking:

The shape of the array changes.

8.4 Visual Interpretation

A useful conceptual visualization is:

$$\begin{bmatrix} Particle \\ Particle \\ Particle \\ Particle \\ Padding \\ Padding \\ Padding \end{bmatrix}$$

Mask:

$$\begin{bmatrix} True \\ True \\ True \\ True \\ False \\ False \\ False \end{bmatrix}$$

Applying the mask:

↓

produces:

$$\begin{bmatrix} Particle \\ Particle \\ Particle \\ Particle \end{bmatrix}$$

Only rows associated with:

True

survive.

Rows associated with:

False

are discarded completely.

9 Understanding Boolean Indexing

Many beginners find the following statement surprising:

```
first_jet[mask]
```

At first glance, it appears strange because:

- `first_jet` contains numerical values.
- `mask` contains Boolean values.

How can a Boolean array be used as an index?

The answer lies in NumPy's Boolean Indexing mechanism.

9.1 How NumPy Interprets the Mask

Suppose:

```
mask =
[True, True, False, True]
```

NumPy examines each element individually.

The rule is:

$$True \rightarrow Keep$$

$$False \rightarrow Discard$$

Therefore:

Mask	Action	Result
True	Keep Row	Included
True	Keep Row	Included
False	Remove Row	Discarded
True	Keep Row	Included

9.2 Internal Selection Process

Conceptually, NumPy performs something similar to:

```
for i in range(139):
    if mask[i] == True:
        keep first_jet[i]
    else:
        ignore first_jet[i]
```

However, NumPy performs this operation internally using optimized C code.

Consequently, Boolean Indexing is:

- Fast
- Memory efficient
- Vectorized
- Easy to read

No explicit Python loop is required.

9.3 Why the Shapes Match

An important requirement exists.

The mask must contain one Boolean value for every row being filtered.

In our case:

$$first_jet.shape = (139, 4)$$

and:

$$mask.shape = (139,)$$

There is exactly one Boolean value for every particle position.

Therefore NumPy knows precisely which rows should be retained.

If the dimensions were incompatible, NumPy would raise an error.

9.4 Inspecting the Filtered Jet

The notebook prints:

```
print(jet_filtered)
```

The resulting array contains:

18

rows.

Every row corresponds to a real particle.

Notice that the padded rows:

[0, 0, 0, 0]

have disappeared entirely.

This confirms that Boolean Masking physically removes padding from the dataset.

10 Physics Interpretation

Up to this point, we have discussed masking from a computational perspective.

However, the ultimate purpose of masking is physical rather than computational.

The padded rows do not represent real particles.

They were introduced solely to satisfy the fixed-size storage requirements of machine learning systems.

Consequently:

$$[0, 0, 0, 0]$$

is not a physical particle.

It is simply a placeholder.

10.1 Why Physicists Remove Padding

When studying the internal structure of jets, we are interested only in genuine constituents.

Including padded rows would introduce:

- Artificial particles
- Incorrect particle counts
- Distorted distributions
- Biased statistical quantities

Therefore padding must be removed before many types of analysis.

10.2 Applications of Boolean Masking

Boolean Masking is widely used in particle physics and machine learning.

Examples include:

- Particle-level feature engineering
- Jet constituent analysis
- Multiplicity studies
- Graph Neural Networks
- Event reconstruction
- Statistical analysis of particles

Whenever only genuine particles are required, Boolean Masking provides a natural solution.

10.3 Limitation of Boolean Masking

Although Boolean Masking is extremely useful, it introduces one important consequence. The shape of the data changes.

Originally:

$(139, 4)$

After masking:

$(18, 4)$

This behavior is ideal when we wish to remove padding.

However, some scientific and machine learning workflows require the original tensor shape to remain unchanged.

In such situations, a different approach is needed.

That approach is provided by:

NumPy Masked Arrays

Instead of removing padded values, Masked Arrays preserve the original structure while marking invalid values as hidden.

In the next section, we will explore how NumPy Masked Arrays work and compare them with Boolean Masking.

11 NumPy Masked Arrays (`numpy.ma`)

In the previous sections, we used Boolean Masking to remove padded particles from the dataset.

Boolean Masking works by physically discarding rows associated with:

False

values in the particle mask.

As a result:

(139, 4)

became:

(18, 4)

for the first jet.

This approach is extremely useful when only real particles are required.

However, changing the shape of the data is not always desirable.

Many scientific and machine learning workflows require tensors to maintain fixed dimensions.

For example:

- Neural Networks
- Convolutional Neural Networks
- Transformer Architectures
- Batch Processing Pipelines
- Scientific Simulations

often assume that every observation has the same shape.

Removing rows would violate this requirement.

NumPy therefore provides another masking mechanism:

`numpy.ma`

which introduces the concept of a:

Masked Array

Instead of removing invalid values, Masked Arrays preserve the original data structure while marking selected values as hidden.

12 Creating a Masked Array

The first masked array in the notebook is created using:

```
first_masked_jet =  
ma.masked_where(  
    first_jet == 0,  
    first_jet  
)
```

Although compact, this statement performs several operations internally.

To understand it properly, we will analyze it step by step.

12.1 Step 1: Defining the Masking Condition

The first argument is:

```
first_jet == 0
```

This expression creates a Boolean matrix.

Every feature value is compared with zero.

Real particle values become:

False

because they are not equal to zero.

Padding values become:

True

because they are equal to zero.

For example:

[2.68, 0.35, 4.74, 22]

becomes:

[*False*, *False*, *False*, *False*]

while:

[0, 0, 0, 0]

becomes:

[True, True, True, True]

The resulting Boolean matrix has shape:

(139, 4)

exactly matching the shape of the original jet.

12.2 Step 2: Passing the Original Data

The second argument is:

```
first_jet
```

This is the numerical data that we wish to preserve.

Unlike Boolean Masking, the data itself will not be removed.

Instead, NumPy will create a special object that stores:

- The original data.
- A corresponding Boolean mask.

Both pieces of information remain available internally.

12.3 What masked_where() Actually Does

The function:

```
ma.masked_where(condition, data)
```

can be interpreted as:

Mask every element of `data` where `condition` is True.

In our case:

- Real particle values remain visible.
- Padding values become masked.

Importantly:

The data is not deleted.

It is simply marked as hidden.

This distinction is the key difference between Boolean Masking and Masked Arrays.

13 Internal Structure of a Masked Array

Conceptually, a Masked Array contains two separate components.

13.1 Data Array

$$\begin{bmatrix} 10 & 0.2 & 1.5 & 22 \\ 20 & 0.3 & 1.8 & 211 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

13.2 Mask Array

$$\begin{bmatrix} \textit{False} & \textit{False} & \textit{False} & \textit{False} \\ \textit{False} & \textit{False} & \textit{False} & \textit{False} \\ \textit{True} & \textit{True} & \textit{True} & \textit{True} \\ \textit{True} & \textit{True} & \textit{True} & \textit{True} \end{bmatrix}$$

The Data Array stores the original values.

The Mask Array stores information about which values should be ignored.

13.3 Meaning of the Mask Values

The Boolean values inside the mask have a special interpretation.

Mask Value	Meaning
False	Visible / Valid Data
True	Hidden / Invalid Data

Table 4: Meaning of values inside a Masked Array mask.

Notice that this convention is opposite to the interpretation often used in Boolean Masking.

For Masked Arrays:

True

means:

Hide this value.

while:

False

means:

Keep this value visible.

14 Inspecting the Masked Array

The notebook prints:

```
print(first_masked_jet)
```

The output appears similar to the original jet.

However, the padded rows now appear as:

```
[-- -- -- --]
```

instead of:

```
[0.0 0.0 0.0 0.0]
```

This visual change is extremely important.

It indicates that NumPy now recognizes these entries as masked values.

The values still exist internally.

They are simply hidden from calculations and display operations.

14.1 Why Does NumPy Display --?

Many beginners incorrectly assume that:

```
--
```

means the value has been deleted.

This is not true.

The symbol:

--

is simply NumPy's visual representation for a masked value.

Internally:

- The original value still exists.
- The mask indicates that the value should be ignored.

Therefore:

Masked values are hidden, not removed.

This distinction is fundamental.

15 First Comparison with Boolean Masking

At this point, the difference between the two approaches can already be observed.

Boolean Masking:

```
jet_filtered = first_jet[mask]
```

Result:

(18, 4)

Only real particles remain.

Masked Array:

```
first_masked_jet =  
ma.masked_where(...)
```

Result:

(139, 4)

All rows remain present.

Padding values are merely hidden.

Thus:

Boolean Masking removes data.

while

Masked Arrays preserve data and attach a mask.

This distinction will become even more important in the next section, where we investigate how Masked Arrays preserve the original shape of the dataset and why that property is useful for scientific computing and machine learning.

16 Understanding the Shape of a Masked Array

One of the most important differences between Boolean Masking and Masked Arrays becomes apparent when we inspect the shape of the resulting array.

The notebook evaluates:

```
print(first_jet.shape)
print(first_masked_jet.shape)
```

Output:

```
first_jet shape: (139,4)
first_masked_jet shape: (139,4)
```

At first glance, this result may seem surprising.

After all, we have already identified that only:

18

particle positions contain real particles.

The remaining:

121

positions correspond to padding.

Many students therefore expect the shape to become:

(18, 4)

similar to Boolean Masking.

However, this does not happen.

The shape remains:

(139, 4)

This behavior is intentional and represents the defining characteristic of NumPy Masked Arrays.

16.1 Why the Shape Does Not Change

Recall how the masked array was created:

```
first_masked_jet =
ma.masked_where(
```

```
    first_jet == 0,  
    first_jet  
)
```

The function:

```
masked_where()
```

does not remove any rows.

Instead, it creates:

- The original data array.
- A corresponding Boolean mask.

Since the data itself remains unchanged, the tensor dimensions remain unchanged as well.

Conceptually:

Original Jet

(139, 4)

↓

Attach Mask

↓

Masked Array

(139, 4)

Nothing has been deleted.

The mask simply tells NumPy which values should be ignored.

16.2 Comparing the Two Approaches

The difference can be summarized visually.

Boolean Masking

$(139, 4)$

↓

Remove Padding

↓

$(18, 4)$

Shape changes.

Masked Arrays

$(139, 4)$

↓

Hide Padding

↓

$(139, 4)$

Shape preserved.

This distinction becomes extremely important in machine learning applications.

17 Why Preserving Shape Is Useful

Many scientific workflows require every observation to have identical dimensions.

For example, suppose we wish to train a neural network.

The network expects inputs of the form:

$(batch_size, 139, 4)$

If Boolean Masking is applied:

$$Jet\ 1 \rightarrow (18, 4)$$

$$Jet\ 2 \rightarrow (35, 4)$$

$$Jet\ 3 \rightarrow (72, 4)$$

the shapes become inconsistent.

The jets can no longer be stacked into a single tensor.

This creates significant difficulties for batch processing.

Masked Arrays avoid this problem.

All jets retain the shape:

$$(139, 4)$$

while still allowing padded values to be ignored during calculations.

Consequently:

- Tensor dimensions remain consistent.
- Batch processing remains possible.
- Padding does not influence calculations.

This makes Masked Arrays particularly attractive in scientific computing.

18 Automatic Exclusion of Masked Values

One of the most powerful features of Masked Arrays is that NumPy automatically ignores masked values during many mathematical operations.

For example:

```
np.mean(first_masked_jet[:,0])
```

computes the average transverse momentum:

$$p_T$$

of all particles contained within the jet.

Importantly:

The masked zeros are excluded automatically.

No additional filtering is required.

This behavior is one of the primary reasons Masked Arrays exist.

18.1 Why This Matters

Suppose we calculate:

$$\text{Mean } p_T$$

using the original jet.

The calculation includes:

$$121$$

artificial zero values.

The resulting average becomes artificially small.

Physically, those zeros do not correspond to particles.

Therefore they should not contribute to the calculation.

Masked Arrays solve this problem automatically.

NumPy simply ignores every masked value.

The user does not need to manually remove padding before performing statistical calculations.

19 Visualizing the Padding Mask

A Masked Array internally stores a Boolean mask.

We can visualize this mask directly.

The notebook uses:

```
plt.figure(figsize=(4,8))

plt.imshow(
    first_masked_jet.mask,
    aspect='auto'
)

plt.title(
    "Padding Mask for First Jet"
)
```

```
plt.xlabel("Feature")  
plt.ylabel("Particle Index")  
plt.colorbar(label="Masked?")  
plt.show()
```

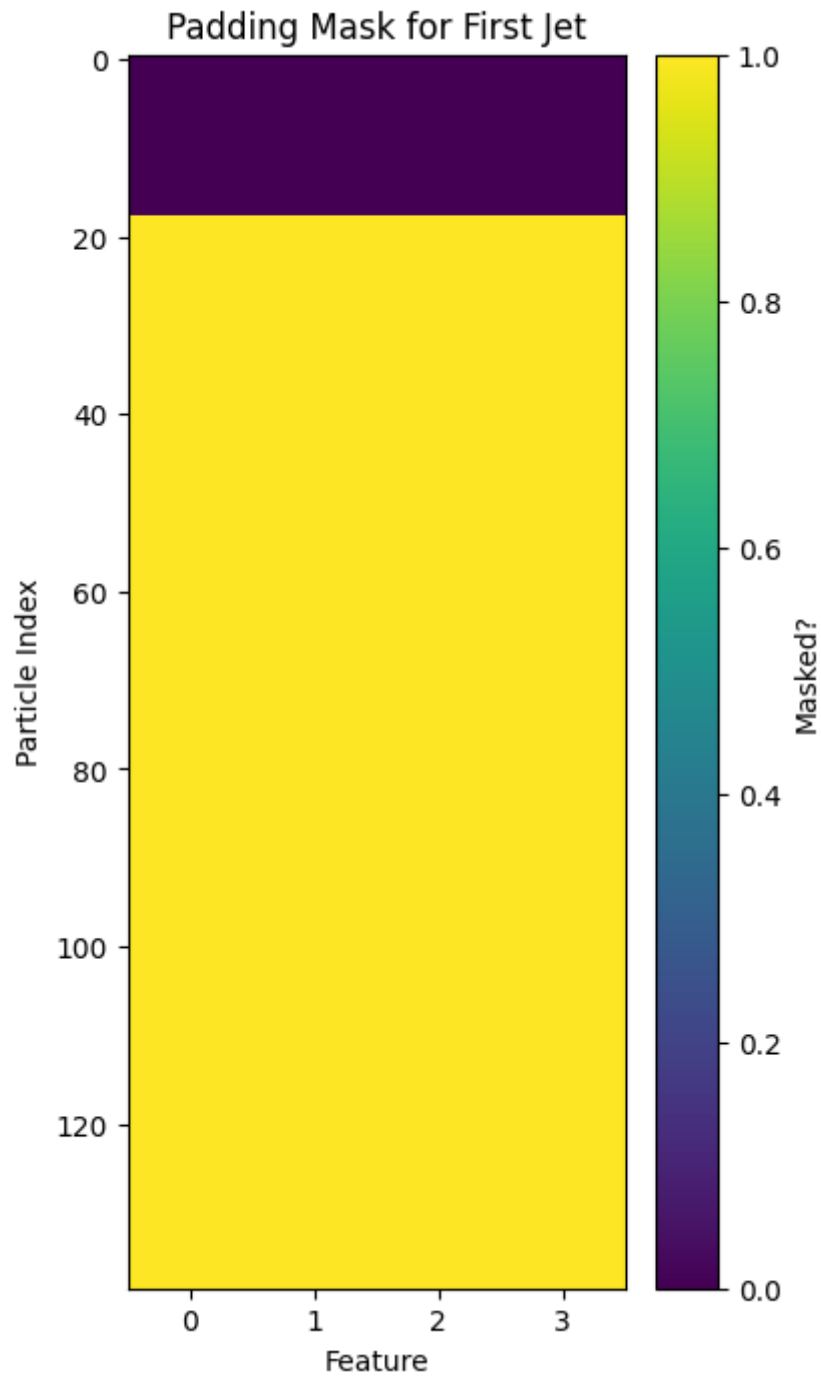


Figure 7: Padding mask visualization for the first jet.

19.1 Interpreting the Heatmap

The heatmap provides a direct visualization of the mask stored inside the Masked Array. Each row corresponds to:

Particle Position

Each column corresponds to:

Particle Feature

The color indicates whether a value is masked.

Conceptually:

Mask Value	Meaning
False	Valid Feature
True	Masked Feature

Table 5: Interpretation of mask values.

The upper portion of the heatmap corresponds to genuine particles.

Most entries appear unmasked.

The lower portion corresponds to padded particle positions.

These rows appear fully masked.

19.2 The Transition Region

A particularly interesting feature of the heatmap is the transition point.

The first:

18

rows correspond to real particles.

These rows contain meaningful physical information.

Beginning at row:

19

the heatmap suddenly changes.

All remaining rows are completely masked.

This transition visually reveals the exact point at which padding begins.

Conceptually:

Particle 1

Particle 2

$\vdots$

Particle 18

Padding Begins

Padding

Padding

Padding

This provides a much clearer picture of the internal structure of the jet than raw numerical output alone.

19.3 Why This Visualization Is Useful

The heatmap demonstrates several important concepts simultaneously:

- The mask has the same shape as the original data.
- Real particles remain visible.
- Padded particles become masked.
- Tensor dimensions remain unchanged.
- Masked Arrays preserve structural information.

This visualization therefore provides strong intuition regarding how Masked Arrays differ from Boolean Masking.

20 Preparing for Quantitative Comparison

At this point we have examined both masking approaches:

- Boolean Masking
- Masked Arrays

We have seen that:

- Boolean Masking physically removes padding.
- Masked Arrays preserve the original tensor shape.
- Both techniques identify the same padded particle positions.

The next question is whether both approaches produce the same physical results.

To answer this question, we will compare:

- Particle counts
- Mean transverse momentum
- Statistical calculations

for both masking methods.

This comparison will reveal that although the internal mechanisms differ substantially, both approaches ultimately recover the same physically meaningful quantities.

21 Comparing Particle Counts Using Both Masking Techniques

Up to this point, we have explored two different approaches for handling padded particles:

- Boolean Masking
- NumPy Masked Arrays

Although the internal mechanisms of these approaches differ significantly, an important question remains:

Do they produce the same physical results?

To answer this question, we begin by comparing the number of particles identified by each method.

Since both techniques are designed to distinguish genuine particles from padded entries, they should ideally recover the same particle count.

This section verifies that expectation.

21.1 Particle Counting Using Boolean Masking

The notebook computes the number of particles using:

```
particle_count_boolean = np.sum(mask)

print(
    f"Total number of Particles in first jet
    after applying Boolean Masking:
    {particle_count_boolean}"
)
```

Output:

```
Total number of Particles in first jet
after applying Boolean Masking: 18
```

Recall that:

$$mask = (first_jet! = 0).any(axis = 1)$$

contains:

Boolean values.

Each element corresponds to one particle position.

The interpretation is:

$$True \rightarrow Real\ Particle$$

$$False \rightarrow Padding$$

Because NumPy treats:

$$True = 1$$

and

$$False = 0$$

the operation:

```
np.sum(mask)
```

counts the total number of real particles.

Conceptually:

$$[True, True, True, \dots, False]$$

becomes:

$$[1, 1, 1, \dots, 0]$$

and the sum equals:

$$18$$

Therefore:

$$Particle\ Count = 18$$

21.2 Particle Counting Using Masked Arrays

The notebook next computes the particle count using the Masked Array representation.


```
particle_count_ma =  
np.sum(  
~(first_masked_jet.mask)  
.all(axis=1)  
)  
  
print(  
f"Total number of Particles in first jet  
after applying Masked Array:  
{particle_count_ma}"  
)
```

Output:

```
Total number of Particles in first jet  
after applying Masked Array: 18
```

At first glance, this expression appears considerably more complicated.

Let us break it down carefully.

21.3 Understanding the Calculation

Recall that:

```
first_masked_jet.mask
```

returns the Boolean mask attached to the Masked Array.

Its shape is:

$(139, 4)$

Each element indicates whether a feature is masked.

For a real particle:

$[False, False, False, False]$

For a padded particle:

$[True, True, True, True]$

The operation:

```
.all(axis=1)
```

asks:

Are all four features masked?

For real particles:

False

is returned.

For padded particles:

True

is returned.

Consequently:

$(False, False, \dots, True, True)$

is obtained.

Next:

`~`

inverts the values.

Thus:

$False \rightarrow True$

and

$True \rightarrow False$

The resulting array now identifies:

$True \rightarrow Real\ Particle$

$False \rightarrow Padding$

which is identical to the Boolean Masking representation.

Finally:

`np.sum(...)`

counts the True values.

The result is again:

18

particles.

21.4 Comparing the Results

The two methods produce:

Method	Particle Count
Boolean Masking	18
Masked Arrays	18

Table 6: Particle count obtained using both masking techniques.

The agreement is perfect.

This confirms that both techniques correctly identify the same physical particles.

Although the internal implementations differ, both approaches successfully remove the influence of padding.

22 Comparing the Mean Transverse Momentum

Particle counts provide a useful sanity check.

However, the true purpose of masking is not merely to count particles.

The primary objective is to ensure that physical quantities are calculated correctly.

One of the most important particle-level observables is:

$$p_T$$

the transverse momentum.

We therefore compare the mean transverse momentum obtained using:

- No masking
- Boolean Masking
- Masked Arrays

22.1 Mean p_T Without Masking

The notebook computes:

```
np.mean(first_jet[:,0])
```

Output:

```
3.6030017985928127
```

Therefore:

$$\text{Mean } p_T = 3.603$$

At first glance, this may appear reasonable.

However, this value is actually misleading.

The calculation includes:

121

padding rows.

Since those rows contain:

$$p_T = 0$$

they artificially reduce the average.

The resulting mean therefore does not represent the true physical properties of the jet.

22.2 Mean p_T Using Boolean Masking

The notebook next evaluates:

```
np.mean(  
jet_filtered[:,0]  
)
```

Output:

```
27.823180555800054
```

Thus:

$$\text{Mean } p_T = 27.823$$

This value is dramatically larger.

Why?

Because the padded rows have been removed completely.

Only genuine particles contribute to the average.

The resulting value therefore reflects the actual transverse momentum distribution of the jet.

22.3 Mean p_T Using Masked Arrays

Finally:

```
np.mean(
    first_masked_jet[:,0]
)
```

Output:

```
27.823180555800054
```

Again:

$$\text{Mean } p_T = 27.823$$

The result matches the Boolean Masking calculation exactly.

This agreement demonstrates that Masked Arrays successfully ignore padded values during statistical calculations.

23 Why Does the Mean Change So Dramatically?

The difference between:

3.603

and

27.823

may initially seem surprising.

However, the explanation is straightforward.

The first jet contains:

18

real particles.

It also contains:

121

padding rows.

When padding is included:

121

additional zeros enter the calculation.

These zeros contribute nothing to the numerator but increase the denominator substantially.

Consequently, the average becomes much smaller.

Mathematically:

$$Mean = \frac{\sum p_T}{139}$$

instead of:

$$Mean = \frac{\sum p_T}{18}$$

The second expression correctly represents the physics.

The first expression does not.

24 Physical Interpretation

The comparison demonstrates an important principle in particle physics data analysis.

Padding is not physical.

Padding exists solely because computers require fixed-size data structures.

Therefore:

- Padding should never contribute to observables.
- Padding should never influence distributions.
- Padding should never alter physical conclusions.

Masking ensures that only genuine particles participate in calculations.

As a result:

- Particle counts remain correct.
- Momentum measurements remain meaningful.
- Statistical quantities remain physically interpretable.

25 Summary of Results

The results obtained for the first jet are summarized below.

Quantity	Value
Particle Count (Boolean Masking)	18
Particle Count (Masked Arrays)	18
Mean p_T Without Masking	3.603
Mean p_T Using Boolean Masking	27.823
Mean p_T Using Masked Arrays	27.823

Table 7: Comparison of physical quantities obtained using different masking approaches.

The equality of the masked results demonstrates that both approaches correctly isolate the physically meaningful constituents of the jet.

Although they operate differently internally, they ultimately recover the same physical information.

The next section extends these ideas from a single jet to the entire dataset containing 100000 jets.

26 Extending Boolean Masking to the Entire Dataset

So far, our analysis has focused on a single jet.

Studying one jet is useful for understanding the mechanics of masking, but it does not reveal how masking behaves when applied to an entire dataset.

The QG dataset contains:

100000

jets.

Each jet contains up to:

139

particle positions.

Therefore, the complete feature tensor has shape:

(100000, 139, 4)

At this scale, manually inspecting individual jets becomes impossible.

Instead, we must use vectorized NumPy operations capable of processing the entire dataset simultaneously.

The goal of this section is to determine:

- How many particles exist in each jet.
- The distribution of particle counts across the dataset.
- How masking scales from a single jet to large datasets.

27 Constructing a Dataset-Level Particle Mask

Previously, for a single jet, we created:

```
mask =
(first_jet != 0).any(axis=1)
```

The same idea can be extended to all jets simultaneously.

The notebook computes:

```
particles_per_jet =
(
(X != 0)
```



```
.any(axis=2)
.sum(axis=1)
)
```

Although this expression appears compact, it performs multiple operations internally.

We will analyze it carefully.

27.1 Step 1: Comparing the Entire Dataset with Zero

The first operation is:

```
X != 0
```

Recall that:

$$X.shape = (100000, 139, 4)$$

Every feature value of every particle in every jet is compared with zero.

The resulting Boolean tensor therefore has shape:

$$(100000, 139, 4)$$

The shape remains unchanged.

Only the numerical values are replaced by:

$$True$$

and

$$False$$

values.

Conceptually:

$$Real\ Feature \rightarrow True$$

$$Padding\ Feature \rightarrow False$$

At this stage, every feature in the entire dataset has been classified as either valid or padded.

27.2 Step 2: Reducing Feature-Level Information

The next operation is:

```
(X != 0).any(axis=2)
```

Recall that:

$$axis = 2$$

corresponds to the feature dimension.

For each particle, the operation asks:

Does at least one feature contain information?

If yes:

True

If no:

False

The feature dimension is collapsed.

Consequently:

(100000, 139, 4)

becomes:

(100000, 139)

The resulting tensor can be interpreted as a dataset-wide particle mask.

Each element now indicates whether a particle position is occupied.

27.3 Interpreting the Dataset-Level Mask

The resulting mask can be visualized conceptually as:

$$\begin{bmatrix} True & True & True & \cdots & False \\ True & True & \cdots & False & False \\ True & True & True & True & True \\ \vdots & \vdots & \vdots & \vdots & \vdots \end{bmatrix}$$

Rows correspond to jets.

Columns correspond to particle positions.

The interpretation is:

- True = Real Particle
- False = Padding

Unlike the single-jet mask discussed earlier, this mask describes the occupancy pattern of the entire dataset simultaneously.

27.4 Step 3: Counting Particles in Each Jet

The final operation is:

```
.sum(axis=1)
```

The summation occurs across particle positions.

Recall:

$$True = 1$$

$$False = 0$$

Therefore:

$$sum(axis = 1)$$

counts the number of occupied particle positions within each jet.

The result is:

$$particles_per_jet$$

which has shape:

$$(100000,)$$

28 Understanding the Output Shape

The notebook prints:

```
print(  
particles_per_jet.shape  
)
```

Output:

```
(100000,)
```

This means:

- One value exists for every jet.
- Each value represents the number of particles contained within that jet.

For example:

particles_per_jet[0]

contains the particle count for Jet 0.

Similarly:

particles_per_jet[99999]

contains the particle count for the final jet.

Thus, the entire dataset has been reduced to a one-dimensional summary describing particle multiplicities.

29 Visualizing the Particle Count Distribution

Once the particle count of every jet has been determined, the next step is to study the overall distribution.

The notebook creates the histogram:

```
plt.hist(  
particles_per_jet,  
bins=50  
)  
  
plt.xlabel(  
"Particles per Jet"  
)  
  
plt.ylabel(  
"Frequency"  
)
```

```
plt.title(
    "Jet Constituent Distribution"
)

plt.show()
```

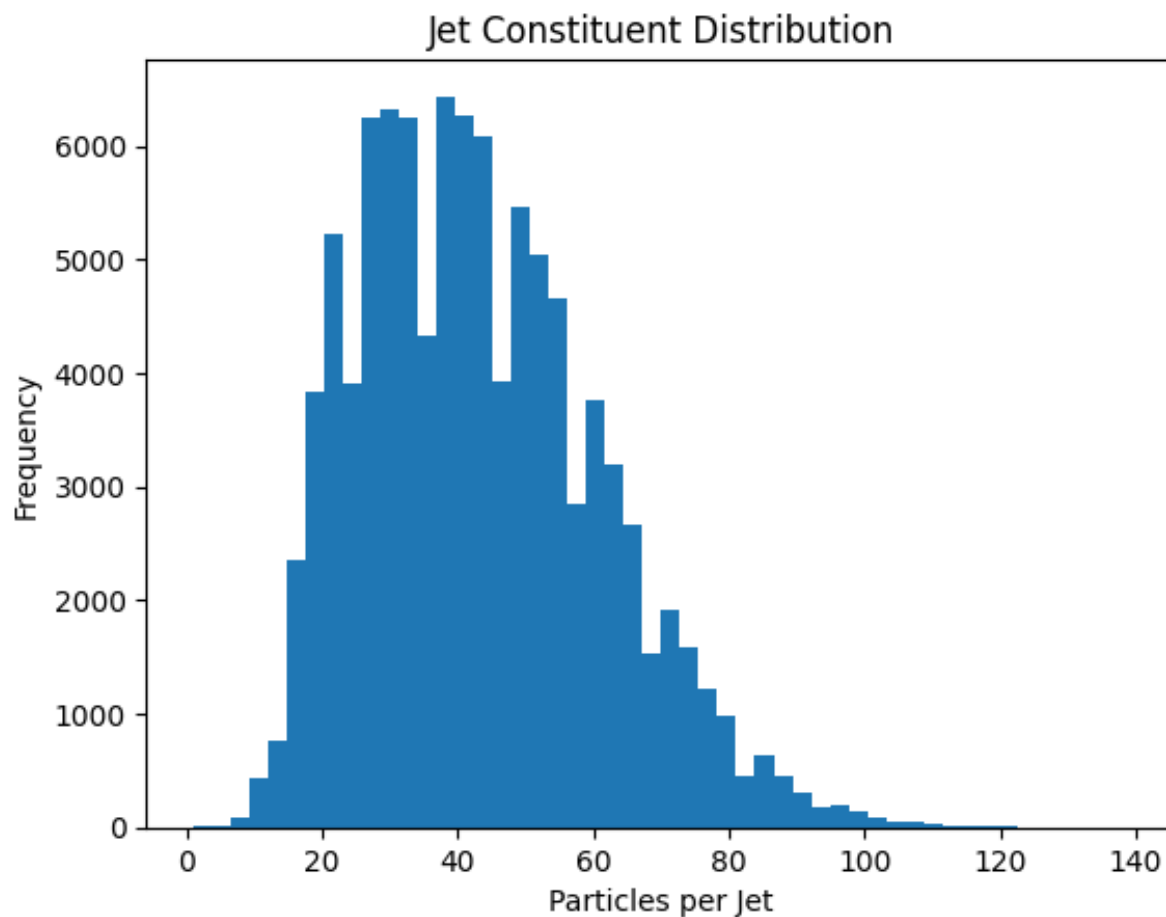


Figure 8: Distribution of particle counts across all jets in the dataset.

30 Interpreting the Histogram

The histogram summarizes the particle content of all:

100000

jets.

The horizontal axis represents:

Particles Per Jet

The vertical axis represents:

Frequency

which indicates the number of jets falling within each particle-count interval.

Every bar therefore answers the question:

How many jets fall within this particle-multiplicity range?

Because the figure is a histogram, each bar represents a range of particle counts rather than a single exact value.

For example:

- Bars near 20 correspond to jets with particle multiplicities around 20.
- Bars near 50 correspond to jets with particle multiplicities around 50.
- Bars near 139 correspond to jets approaching the maximum storage capacity of the dataset.

The distribution exhibits a clear peak around 35–40 particles, indicating that this multiplicity range occurs most frequently within the dataset.

Beyond the peak region, the frequency gradually decreases as the particle count increases, producing a long tail that extends toward larger multiplicities.

Only a relatively small fraction of jets contain very large numbers of particles.

The histogram is therefore positively skewed, with low-to-moderate multiplicity jets occurring more frequently than high-multiplicity jets.

Consequently, the histogram provides a global view of particle multiplicity across the entire dataset.

31 Physics Interpretation of Particle Multiplicity

The number of particles contained within a jet is often referred to as the:

Particle Multiplicity

or

Jet Constituent Count

Particle multiplicity is an important observable in high-energy physics.

Jets with larger multiplicities typically correspond to more complex particle showers.

Jets with smaller multiplicities correspond to simpler fragmentation patterns.

Consequently, particle multiplicity can provide information about the internal structure of jets.

The histogram demonstrates that the dataset contains jets with a broad range of constituent multiplicities, reflecting the diversity of particle shower topologies produced during high-energy collisions.

In quark–gluon classification tasks, constituent multiplicity is often one of the features used to distinguish between different jet types.

32 What the Histogram Reveals About Padding

The histogram also reveals something important about padding.

Recall that every jet is stored using:

139

particle positions.

However, many jets contain substantially fewer than 139 particles.

Therefore:

Many particle positions remain unused.

These unused positions must be filled with padding.

Consequently, the histogram indirectly provides information about the amount of padding present within the dataset.

Jets appearing far below:

139

require substantial padding.

Jets approaching:

139

require relatively little padding.

Thus, the histogram provides insight into both:

- Physical jet structure.
- Dataset storage characteristics.

33 Why This Analysis Matters

The particle count distribution demonstrates an important principle.

Masking is not merely a tool for inspecting a single jet.

The same concepts scale naturally to datasets containing hundreds of thousands of observations.

Using a single vectorized NumPy expression, we have:

- Identified all real particles.
- Ignored all padding.
- Counted particles in every jet.
- Constructed a dataset-wide distribution.

This illustrates one of the major strengths of NumPy:

Large-scale operations can be performed without explicit Python loops.

34 Preparing for Jet-Level Filtering

The analysis performed so far preserves the original dataset structure.

We counted particles without physically removing any rows.

However, there are situations where we may wish to eliminate padding entirely and obtain variable-length particle lists for individual jets.

Doing so introduces an important distinction:

- Dataset-Level Masking
- Jet-Level Masking

The next section explores this distinction and explains why filtering an entire dataset is fundamentally different from filtering a single jet.

35 Dataset-Level Masking vs Jet-Level Masking

In the previous section, Boolean Masking was applied to the entire dataset in order to count particles within every jet.

The operation:

```
(X != 0).any(axis=2)
```

produced a dataset-wide particle mask while preserving the original tensor structure.

This approach is extremely efficient and well suited for large-scale statistical analysis.

However, there are situations where simply counting particles is not sufficient.

In many physics and machine learning applications, we wish to work directly with the actual particles contained within each jet.

In such cases, the padded rows must be removed entirely.

This introduces an important distinction:

- Dataset-Level Masking
- Jet-Level Masking

Although both approaches use the same masking principles, they serve different objectives and produce fundamentally different outputs.

36 Dataset-Level Masking

Dataset-Level Masking operates on the entire dataset simultaneously.

Recall:

$$X.shape = (100000, 139, 4)$$

The operation:

```
complete_mask =  
(X != 0).any(axis=2)
```

produces:

$$(100000, 139)$$

Each row corresponds to a jet.

Each column corresponds to a particle position.

The mask therefore answers:

Does this particle position contain a real particle?

for every particle in every jet.

Importantly:

The dataset structure remains intact.

No particles are removed.

No jet boundaries are altered.

The mask simply provides information about particle occupancy.

36.1 When Dataset-Level Masking Is Useful

Dataset-Level Masking is appropriate when the objective is to analyze the dataset while preserving its original tensor structure.

Examples include:

- Counting particles per jet.
- Measuring padding frequency.
- Constructing multiplicity distributions.
- Computing dataset-wide statistics.
- Preparing fixed-size tensors for machine learning.

For example:

```
particle_counts =  
complete_mask.sum(axis=1)
```

returns the number of particles in every jet without modifying the dataset itself.

This operation is highly efficient because it is fully vectorized.

37 Jet-Level Masking

Dataset-Level Masking identifies padded particles.

Jet-Level Masking goes one step further.

It removes them.

The notebook performs:

```

filtered_jets =
[
jet[(jet != 0).any(axis=1)]
for jet in X
]

```

At first glance, this expression appears similar to previous masking operations.

However, it performs a fundamentally different task.

Instead of preserving the original tensor structure, it creates a collection of jets containing only genuine particles.

37.1 Breaking Down the Expression

The operation can be separated into two components.

First:

```
(jet != 0).any(axis=1)
```

creates a particle mask for a single jet.

This is identical to the procedure discussed earlier.

The result identifies:

True → Real Particle

False → Padding

Next:

```
jet[mask]
```

applies Boolean Indexing and removes padded rows.

The output contains only genuine particles.

Finally:

```
for jet in X
```

repeats the process for every jet in the dataset.

The resulting object:

filtered_jets

contains 100000 filtered jets.

38 Inspecting the Filtered Jets

The notebook prints:

```
print(
    filtered_jets[7165].shape
)

print(
    filtered_jets[0].shape
)
```

Output:

```
(139, 4)
(18, 4)
```

These shapes reveal an extremely important property.

Different jets contain different numbers of particles.

For example:

$$\text{Jet } 0 \rightarrow (18, 4)$$

while:

$$\text{Jet } 7165 \rightarrow (139, 4)$$

This means:

- Jet 0 contains 18 particles.
- Jet 7165 contains 139 particles.

The filtered jets therefore no longer share a common shape.

38.1 Why the Shapes Differ

This behavior is not a consequence of NumPy.

It is a consequence of physics.

Different jets naturally contain different numbers of particles.

For example:

$$\text{Jet } A \rightarrow 18 \text{ Particles}$$

Jet B $\rightarrow$ 42 Particles

Jet C $\rightarrow$ 75 Particles

Jet D $\rightarrow$ 139 Particles

Once padding is removed, these differences become visible.

The resulting shapes therefore reflect the true physical structure of the data.

39 Variable-Length Outputs

A major consequence of Jet-Level Masking is the creation of variable-length outputs.

Originally:

$$X.shape = (100000, 139, 4)$$

Every jet had identical dimensions.

After filtering:

$$Jet\ 0 = (18, 4)$$

$$Jet\ 1 = (25, 4)$$

$$Jet\ 2 = (33, 4)$$

$$Jet\ 3 = (12, 4)$$

$\vdots$

Each jet now contains a different number of rows.

This creates a collection of variable-length arrays.

Such structures are extremely common in particle physics because physical systems rarely contain identical numbers of particles.

39.1 Why a Python List Is Required

The notebook evaluates:

```
len(filtered_jets)
```

Output:

```
100000
```

Notice that:

filtered_jets

is a Python list.

A standard NumPy array cannot store:

(18, 4)

(25, 4)

(33, 4)

simultaneously because NumPy requires rectangular dimensions.

The list structure solves this problem by allowing each jet to maintain its own shape.

Conceptually:

```
filtered_jets
|
|-- Jet 0 -> (18,4)
|
|-- Jet 1 -> (25,4)
|
|-- Jet 2 -> (33,4)
|
|-- Jet 3 -> (12,4)
|
...
```

40 Why We Cannot Simply Write

A common question is:

```
X_filtered =
X[complete_mask]
```

Why not filter the entire dataset directly?

The answer lies in how Boolean Indexing behaves.

40.1 What Actually Happens

Recall:

$$complete_mask.shape = (100000, 139)$$

Every True value identifies a particle.

When Boolean Indexing is applied:

```
X[complete_mask]
```

NumPy collects every particle associated with a True value.

The result becomes:

$$(total_real_particles, 4)$$

Conceptually:

Jet 0

Jet 1

Jet 2

Jet 3

all become merged into a single giant particle collection.

40.2 Information Is Lost

The major problem is that jet boundaries disappear.

After filtering:

We know which particles exist.

but

We no longer know which jet they came from.

This information is essential for most physics analyses.

Therefore:

$$X[\textit{complete_mask}]$$

is generally unsuitable when jet-level structure must be preserved.

41 Dataset-Level vs Jet-Level Masking

The two approaches can now be compared directly.

Property	Dataset-Level Masking	Jet-Level Masking
Preserves Tensor Shape	Yes	No
Removes Padding	No	Yes
Maintains Fixed Dimensions	Yes	No
Produces Variable-Length Outputs	No	Yes
Suitable for Histograms	Excellent	Limited
Suitable for Particle Lists	No	Excellent
Preserves Jet Boundaries	Yes	Yes
Efficient for Large Datasets	Excellent	Moderate

Table 8: Comparison of dataset-level and jet-level masking.

42 Key Takeaway

Dataset-Level Masking and Jet-Level Masking solve different problems.

Dataset-Level Masking is designed for large-scale statistical analysis while preserving the original tensor structure.

Jet-Level Masking is designed for extracting the genuine particles contained within each jet.

Neither approach is universally better.

The correct choice depends on the objective of the analysis.

If the goal is:

- Counting particles,

- Building histograms,
- Computing dataset-wide statistics,

Dataset-Level Masking is usually preferable.

If the goal is:

- Studying individual jet constituents,
- Removing padding,
- Creating variable-length particle lists,

Jet-Level Masking becomes necessary.

The next section explores how the same dataset-wide analysis can be performed using NumPy Masked Arrays while preserving the original tensor structure.

43 Applying Masked Arrays to the Entire Dataset

In the previous section, we extended Boolean Masking from a single jet to the entire dataset.

We constructed a dataset-wide particle mask and used it to compute particle multiplicities for all:

100000

jets.

However, Boolean Masking is not the only technique capable of performing dataset-level analysis.

NumPy Masked Arrays can also be applied directly to the complete dataset.

This approach provides several advantages:

- Original tensor dimensions are preserved.
- Padding is automatically ignored during calculations.
- Dataset-wide statistics become easier to compute.
- Fixed-size tensor structures remain intact.

In this section, we investigate how Masked Arrays behave when applied to the full QG dataset.

44 Creating a Dataset-Wide Masked Array

The notebook creates:

```
X_masked =
ma.masked_where(
X == 0,
X
)
```

This operation is identical in spirit to the masked array created for the first jet.

The difference is that the operation now acts on the complete dataset.

Recall:

$$X.shape = (100000, 139, 4)$$

Therefore:

$$X_masked.shape = (100000, 139, 4)$$

The shape remains unchanged.

The dataset still contains:

- 100000 jets
- 139 particle positions
- 4 particle features

The only difference is that padded entries are now marked as masked.

44.1 What Does `X_masked` Contain?

Conceptually, the masked dataset contains two components:

Data Tensor

$$(100000, 139, 4)$$

containing the original numerical values.

Mask Tensor

$$(100000, 139, 4)$$

containing Boolean values indicating which entries should be ignored.

Thus:

$$X_masked$$

preserves every original value while simultaneously storing information about padding.

Nothing is removed.

Everything remains available internally.

45 Computing Particle Counts for Every Jet

The notebook next computes:

```

particles_per_jet =
np.sum(
~(
X_masked.mask
).all(axis=2),
axis=1
)

```

At first glance, this expression appears considerably more complicated than the Boolean Masking equivalent.

However, the underlying logic is identical.

45.1 Step 1: Accessing the Mask Tensor

The operation begins with:

```
X_masked.mask
```

This returns the Boolean mask stored inside the Masked Array.

Its shape is:

(100000, 139, 4)

Each Boolean value indicates whether a feature is masked.

The interpretation is:

False $\rightarrow$ *Valid Feature*

True $\rightarrow$ *Masked Feature*

45.2 Step 2: Determining Whether a Particle Is Fully Masked

Next:

```
.all(axis=2)
```

is applied.

Recall:

$$axis = 2$$

corresponds to the feature dimension.

The operation asks:

Are all four features masked?

For a real particle:

$[False, False, False, False]$

the answer is:

$False$

For a padded particle:

$[True, True, True, True]$

the answer is:

$True$

The resulting shape becomes:

$(100000, 139)$

One Boolean value now exists for every particle position.

45.3 Step 3: Inverting the Logic

The notebook next applies:

$\sim$

which inverts the Boolean values.

Thus:

$True \rightarrow False$

$False \rightarrow True$

The interpretation becomes:

$$True \rightarrow Real\ Particle$$

$$False \rightarrow Padding$$

which now matches the Boolean Masking representation discussed earlier.

45.4 Step 4: Counting Particles

Finally:

```
np.sum(..., axis=1)
```

counts the True values across particle positions.

The result is:

$$particles_per_jet$$

with shape:

$$(100000,)$$

Exactly one particle count is produced for every jet.

46 Inspecting the Results

The notebook prints:

```
print(  
particles_per_jet.shape  
)
```

Output:

```
(100000,)
```

This confirms that the operation successfully computes particle multiplicities for all jets.

The notebook also prints:

```
print(  
particles_per_jet[7165]  
)
```

Output:

139

This indicates that Jet 7165 contains:

139

real particles.

Therefore:

Jet 7165 requires no padding.

Every available particle position is occupied.

47 Computing the Mean Transverse Momentum of Every Jet

One of the greatest strengths of Masked Arrays is their ability to perform statistical calculations while automatically ignoring masked values.

The notebook computes:

```
mean_pt_per_jet =
np.mean(
X_masked[:, :, 0],
axis=1
)
```

This single statement computes:

$Mean\ p_T$

for every jet in the dataset.

47.1 Understanding the Slice

The expression:

```
X_masked[:, :, 0]
```

selects:

p_T

for every particle in every jet.

Recall:

$$\text{Feature } 0 = p_T$$

Therefore:

$$X_{\text{masked}}[:, :, 0]$$

has shape:

$$(100000, 139)$$

Rows correspond to jets.

Columns correspond to particle positions.

47.2 Applying the Mean

The operation:

```
np.mean(..., axis=1)
```

computes the average across particle positions.

Consequently:

$$139$$

particle positions are reduced to:

$$1$$

average value per jet.

The resulting shape becomes:

$$(100000,)$$

48 Inspecting the Mean p_T Results

The notebook prints:

```
print(
    mean_pt_per_jet.shape
)
```


Output:

```
(100000,)
```

This confirms that one mean transverse momentum value exists for every jet.

The notebook further prints:

```
print(  
mean_pt_per_jet[0]  
)
```

Output:

```
27.823180555800054
```

This value is identical to the mean transverse momentum computed previously for the first jet using both:

- Boolean Masking
- Masked Arrays

The agreement provides an important consistency check.

49 Why Masked Arrays Are Powerful for Dataset-Wide Analysis

At this point, an important advantage becomes apparent.

The original dataset shape remains:

```
(100000, 139, 4)
```

throughout the entire analysis.

No rows were removed.

No loops were required.

No variable-length outputs were created.

Yet:

- Padding was ignored.
- Particle counts were computed.
- Mean transverse momenta were calculated.
- Statistical quantities remained physically meaningful.

This is precisely what makes Masked Arrays attractive for large-scale scientific computing.

49.1 Automatic Statistical Handling

Many statistical functions automatically recognize masked values.

Examples include:

- Mean
- Sum
- Minimum
- Maximum
- Variance
- Standard Deviation

Consequently, analysts can focus on physics rather than manually removing padding before every calculation.

This leads to cleaner and more readable code.

50 Dataset-Wide Comparison

At the dataset level, the distinction between the two approaches becomes especially clear.

Boolean Masking primarily serves as a:

Data Selection Technique

Masked Arrays primarily serve as a:

Data Representation Technique

Boolean Masking removes data.

Masked Arrays preserve data while attaching information about which values should be ignored.

Both approaches solve the padding problem.

However, they are optimized for different workflows.

51 Preparing for the Final Comparison

We have now examined:

- Boolean Masking on a single jet
- Masked Arrays on a single jet
- Boolean Masking on the entire dataset
- Masked Arrays on the entire dataset

Both approaches successfully identify genuine particles and eliminate the influence of padding.

The final question is:

Which technique should be used in practice?

The next section provides a comprehensive comparison of Boolean Masking and Masked Arrays, highlighting their strengths, weaknesses, and ideal use cases.

52 Boolean Masking vs Masked Arrays

Throughout this report, we have explored two different approaches for handling padded particles in the QG jet dataset:

- Boolean Masking
- NumPy Masked Arrays

Both techniques solve the same fundamental problem:

How can padded particle positions be ignored without corrupting physical calculations?

Although both approaches ultimately produce the same physical results, they operate in very different ways and are designed for different types of analysis.

This chapter summarizes the similarities and differences between the two techniques and provides practical guidelines for choosing the appropriate approach.

53 A Common Objective

Before discussing differences, it is important to recognize that both techniques share the same objective.

Both methods attempt to distinguish:

- Genuine particles
- Artificial padding

In the QG dataset, padded particle positions are represented by:

$$[0, 0, 0, 0]$$

rows.

Including these rows in calculations produces incorrect physical quantities.

Therefore:

- Particle counts become inaccurate.
- Mean transverse momentum becomes biased.
- Statistical distributions become distorted.

Both Boolean Masking and Masked Arrays eliminate these problems.

The difference lies in how they achieve this goal.

54 Core Philosophy of Each Approach

Although the outputs often appear similar, the underlying philosophies are fundamentally different.

54.1 Boolean Masking

Boolean Masking is a:

Data Selection Technique

The philosophy is simple:

Remove everything that should not be used.

The operation:

```
jet_filtered =  
jet[mask]
```

physically removes padded rows.

After filtering:

$(139, 4)$

becomes:

$(18, 4)$

Only genuine particles remain.

Padding disappears completely.

54.2 Masked Arrays

Masked Arrays follow a different philosophy.

They are a:

Data Representation Technique

Instead of removing unwanted values, they preserve the original structure and attach information describing which entries should be ignored.

Conceptually:

Keep everything, but mark invalid values.

Thus:

$(139, 4)$

remains:

$(139, 4)$

while padded values become masked.

Nothing is deleted.

55 Structural Differences

The most obvious difference between the two approaches concerns array shape.

55.1 Boolean Masking

Before masking:

$(139, 4)$

After masking:

$(18, 4)$

Shape changes.

55.2 Masked Arrays

Before masking:

$(139, 4)$

After masking:

$(139, 4)$

Shape preserved.

This distinction affects how the data can be used in later stages of analysis.

56 Effect on Memory Usage

Boolean Masking creates a new array containing only selected elements.

Consequently:

- Fewer rows are stored.
- Memory usage may decrease.
- Data becomes more compact.

Masked Arrays preserve the complete dataset and additionally store a mask tensor.

Consequently:

- Original data remains.
- Additional mask information is stored.
- Memory requirements may increase slightly.

For very large datasets, this distinction can become relevant.

57 Effect on Statistical Calculations

One of the most important observations from the notebook was:

$$\text{Mean } p_T = 27.823$$

for both masking approaches.

This demonstrates that both methods successfully exclude padding from calculations.

However, the mechanism differs.

57.1 Boolean Masking

The calculation:

```
np.mean(  
jet_filtered[:,0]  
)
```

works because the padded rows were removed before the mean was computed.

The statistical function never sees the padded values.

57.2 Masked Arrays

The calculation:

```
np.mean(  
    first_masked_jet[:,0]  
)
```

works because NumPy automatically ignores masked entries.

The padded values remain present internally but are excluded from the calculation.

Thus both methods produce identical physical results through different mechanisms.

58 Machine Learning Perspective

Machine learning systems often require fixed-size tensors.

Consider a batch of jets.

Suppose:

$$\text{Jet 1} = (18, 4)$$

$$\text{Jet 2} = (35, 4)$$

$$\text{Jet 3} = (72, 4)$$

These arrays cannot be stacked easily because their shapes differ.

Consequently, Boolean Masking can create difficulties for machine learning pipelines.

Masked Arrays avoid this issue.

Every jet remains:

$$(139, 4)$$

allowing fixed-size tensor structures to be preserved.

For this reason, Masked Arrays are often more convenient for dataset preparation and preprocessing.

59 Particle Physics Perspective

Particle physicists frequently work directly with jet constituents.

Examples include:

- Constituent multiplicity studies.
- Jet shape analysis.
- Fragmentation studies.
- Event reconstruction.

In such situations, padding provides no useful information.

Researchers often prefer Boolean Masking because it produces:

Only real particles.

The resulting arrays correspond directly to physical objects.

This makes subsequent analysis more intuitive.

60 Advantages of Boolean Masking

Boolean Masking offers several benefits.

- Simple and intuitive.
- Physically removes padding.
- Produces compact arrays.
- Easy to interpret.
- Ideal for constituent-level studies.
- Natural representation of physical particle lists.

For many exploratory analyses, Boolean Masking is often the most straightforward choice.

61 Advantages of Masked Arrays

Masked Arrays also provide several powerful features.

- Preserve tensor shape.
- Ideal for fixed-size data structures.
- Automatic handling of invalid values.
- Statistical functions work naturally.

- Convenient for large-scale dataset analysis.
- Integrates well with scientific computing workflows.

These properties make Masked Arrays particularly attractive when working with large multidimensional datasets.

62 Limitations of Boolean Masking

Despite its strengths, Boolean Masking has several limitations.

- Shape changes after filtering.
- Variable-length outputs are created.
- Batch processing becomes more difficult.
- Additional bookkeeping may be required.
- Dataset-wide tensor structures are not preserved.

These limitations become increasingly important as datasets grow larger.

63 Limitations of Masked Arrays

Masked Arrays also have drawbacks.

- Data is not physically removed.
- Slightly more complex to understand.
- Mask management introduces additional overhead.
- Some third-party libraries may not fully support masked arrays.

Consequently, Masked Arrays are not always the optimal choice.

64 Comprehensive Comparison

Property	Boolean Masking	Masked Arrays
Primary Purpose	Data Selection	Data Representation
Removes Padding	Yes	No
Preserves Shape	No	Yes
Creates Variable-Length Outputs	Yes	No
Automatic Statistical Handling	No	Yes
Memory Efficiency	Higher	Slightly Lower
Machine Learning Compatibility	Moderate	Excellent
Particle-Level Analysis	Excellent	Good
Dataset-Level Analysis	Good	Excellent
Ease of Understanding	Excellent	Moderate
Physical Interpretation	Direct	Indirect

Table 9: Comprehensive comparison of Boolean Masking and Masked Arrays.

65 Practical Guidelines

The choice between Boolean Masking and Masked Arrays depends entirely on the objective of the analysis.

Use Boolean Masking when:

- Padding should be removed completely.
- Individual particles are being studied.
- Variable-length particle lists are desired.
- Constituent-level physics analysis is performed.

Use Masked Arrays when:

- Tensor dimensions must remain fixed.
- Dataset-wide statistics are required.
- Machine learning pipelines are involved.
- Automatic exclusion of invalid values is beneficial.

There is no universally superior method.

Both approaches are valuable tools that solve different problems.

66 Final Observation

Perhaps the most important result obtained throughout this notebook is that both techniques recover the same physical information.

For the first jet:

$$Particle\ Count = 18$$

using both methods.

Similarly:

$$Mean\ p_T = 27.823$$

using both methods.

Thus, despite their different implementations, both approaches correctly isolate the genuine constituents of the jet and eliminate the influence of padding.

The choice between them therefore depends not on correctness, but on the requirements of the analysis.

67 Key Takeaways

Throughout this report, we explored how NumPy masking techniques can be used to identify and handle padded particles within the QG jet dataset.

Although the notebook began with a simple objective—understanding masking—it ultimately demonstrated several important concepts related to scientific computing, particle physics data analysis, and machine learning preprocessing.

The following key insights summarize the most important lessons learned from this study.

67.1 Real-World Scientific Datasets Often Contain Padding

One of the first observations made in the notebook was that jets naturally contain different numbers of particles.

For example:

Jet A $\rightarrow$ 18 Particles

Jet B $\rightarrow$ 42 Particles

Jet C $\rightarrow$ 75 Particles

Since machine learning algorithms and numerical libraries typically require fixed-size tensors, the dataset stores every jet using:

$(139, 4)$

particle positions.

When a jet contains fewer than 139 particles, the remaining positions are filled with zeros.

These zero-filled rows are known as:

Padding

Padding is not physical data.

It exists solely to maintain a consistent tensor shape.

Understanding this distinction is essential before performing any meaningful analysis.

67.2 Padding Can Distort Statistical Calculations

A major result obtained in this notebook was the dramatic difference between masked and unmasked calculations.

For the first jet:

$$\text{Mean } p_T = 3.603$$

when padding was included.

After masking:

$$\text{Mean } p_T = 27.823$$

The difference is enormous.

This occurs because padded particle positions contribute large numbers of artificial zeros.

Those zeros reduce averages and distort physical observables.

Consequently:

Ignoring padding can lead to incorrect scientific conclusions.

Any statistical analysis involving padded datasets should therefore account for invalid entries before calculations are performed.

67.3 Boolean Masking Converts Physical Questions into Logical Operations

One of the most elegant ideas demonstrated in the notebook is that identifying real particles can be reduced to a simple logical question.

For each particle:

Does at least one feature contain information?

Using:

```
(first_jet != 0).any(axis=1)
```

NumPy converts this physical question into a Boolean mask consisting of:

True

and

False

values.

This mask provides a compact representation of particle occupancy and serves as the foundation for subsequent filtering operations.

67.4 Boolean Masking Physically Removes Padding

Boolean Masking functions as a data selection technique.

When the particle mask is applied:

```
jet_filtered =
first_jet[mask]
```

all padded rows are removed.

The jet shape changes from:

(139, 4)

to:

(18, 4)

Only genuine particles remain.

This makes Boolean Masking particularly useful for:

- Particle-level studies
- Constituent analysis
- Event reconstruction
- Variable-length particle lists

The resulting arrays correspond directly to physically meaningful objects.

67.5 Masked Arrays Preserve the Original Tensor Structure

Unlike Boolean Masking, Masked Arrays do not remove data.

Instead:

- The original tensor is preserved.
- Invalid values are marked as masked.
- Statistical operations automatically ignore masked entries.

Consequently:

(139, 4)

remains:

$$(139, 4)$$

even after masking.

This property is particularly valuable when working with:

- Fixed-size tensors
- Large multidimensional datasets
- Machine learning pipelines
- Batch processing systems

Shape preservation is one of the defining strengths of Masked Arrays.

67.6 Different Methods Can Produce the Same Physical Result

One of the most important findings of this report is that both masking techniques ultimately recover the same physical information.

For the first jet:

$$Particle\ Count = 18$$

using Boolean Masking.

$$Particle\ Count = 18$$

using Masked Arrays.

Similarly:

$$Mean\ p_T = 27.823$$

for both methods.

This agreement demonstrates that the two approaches are mathematically consistent despite their different implementations.

The choice between them is therefore determined by workflow requirements rather than correctness.

67.7 Dataset-Level and Jet-Level Analysis Are Fundamentally Different

The notebook also highlighted an important distinction between:

- Dataset-Level Masking
- Jet-Level Masking

Dataset-Level Masking preserves the tensor structure and is ideal for:

- Histograms
- Statistical distributions
- Multiplicity studies
- Dataset-wide calculations

Jet-Level Masking removes padding from individual jets and is ideal for:

- Constituent analysis
- Variable-length representations
- Particle-level investigations

Understanding when to use each approach is crucial when analyzing large scientific datasets.

67.8 NumPy Enables Large-Scale Vectorized Processing

A particularly powerful aspect of the notebook was the ability to process:

100000

jets simultaneously using compact vectorized operations.

For example:

```
particles_per_jet =  
((X != 0)  
.any(axis=2)  
.sum(axis=1))
```

computes particle counts for the entire dataset without writing explicit nested loops.

This demonstrates one of NumPy's greatest strengths:

Efficient large-scale numerical computation through vectorization.

Such techniques are essential when working with modern scientific datasets.

67.9 Masking Is More Than a NumPy Feature

Although this notebook focused on NumPy, the underlying concept of masking extends far beyond a single library.

Masking appears throughout:

- Scientific Computing
- Data Analysis
- Machine Learning
- Deep Learning
- Computer Vision
- Natural Language Processing
- Particle Physics

Examples include:

- Attention masks in Transformers
- Missing value handling
- Image segmentation masks
- Event selection in particle physics
- Sequence padding masks

Thus, understanding masking provides knowledge that is applicable across many domains.

67.10 Choosing the Right Masking Strategy Depends on the Problem

There is no universally superior masking technique.

The correct choice depends on the objective of the analysis.

Boolean Masking is generally preferable when:

- Padding should be removed.
- Particle lists are needed.
- Physical constituents are studied directly.

Masked Arrays are generally preferable when:

- Tensor shapes must remain fixed.
- Dataset-wide statistics are required.
- Automatic handling of invalid values is desired.

A strong understanding of both approaches allows researchers to select the most appropriate tool for a given task.

68 Summary of the Learning Journey

Beginning with a single padded jet, we progressively developed an understanding of:

- Dataset structure
- Particle padding
- Boolean masks
- Boolean indexing
- Shape transformations
- Masked Arrays
- Dataset-level masking
- Jet-level masking
- Statistical analysis
- Physical interpretation of masking

What initially appeared to be a simple NumPy technique ultimately revealed important concepts related to both computational efficiency and scientific correctness.

The notebook therefore serves not only as an introduction to NumPy masking but also as an example of how careful data preprocessing is essential for extracting meaningful information from real-world scientific datasets.

69 Conclusion

The primary objective of this notebook was to develop a practical understanding of masking techniques in NumPy and investigate how they can be applied to a real-world scientific dataset.

Rather than studying masking through small artificial examples, the exploration was performed using the QG jet dataset, a particle physics dataset containing:

- 100000 jets
- Up to 139 particles per jet
- Four particle-level features for each particle

This provided an opportunity to observe how masking techniques operate in realistic scenarios where data contains padding and variable-length structures.

Throughout the notebook, masking was not treated merely as a programming feature. Instead, it was examined as a practical solution to an important data representation problem that frequently arises in scientific computing and machine learning.

70 What Was Learned

The investigation began by understanding the structure of the dataset.

A detailed inspection of the first jet revealed that only a small fraction of the available particle positions contained meaningful information.

The remaining positions consisted entirely of zero-valued rows introduced through padding.

Although these padded rows are necessary for maintaining fixed-size tensors, they do not correspond to physical particles.

Consequently, they should not influence statistical calculations or scientific analysis.

This observation motivated the need for masking.

The notebook then demonstrated how a simple logical comparison:

```
(first_jet != 0)
```

can be transformed into a particle-level occupancy mask through:

```
.any(axis=1)
```

This particle mask provides a compact representation of which particle positions contain genuine information and which correspond to padding.

From this point onward, the notebook explored two different masking strategies.

71 Understanding Boolean Masking

The first technique investigated was Boolean Masking.

Boolean Masking treats the mask as a filtering mechanism.

Particle positions associated with:

True

are retained, while positions associated with:

False

are removed.

Applying the particle mask to the first jet transformed the original shape:

(139, 4)

into:

(18, 4)

thereby removing all padded rows.

This demonstrated that Boolean Masking acts as a data selection technique.

Only genuine particles remain in the resulting array.

Because the output directly corresponds to physical particles, Boolean Masking provides an intuitive representation for constituent-level analysis.

The notebook further showed how Boolean Masking can be extended to the full dataset to compute particle multiplicities and construct particle count distributions.

72 Understanding Masked Arrays

The second technique investigated was NumPy Masked Arrays.

Unlike Boolean Masking, Masked Arrays do not remove data.

Instead, they preserve the original tensor structure while attaching a Boolean mask indicating which values should be ignored.

Creating:

```
ma.masked_where(
X == 0,
X
)
```

allowed padded entries to remain present within the dataset while ensuring that statistical operations automatically excluded them.

This behavior illustrated a fundamentally different philosophy.

Rather than removing invalid values, Masked Arrays preserve the complete dataset and separate:

- Data storage
- Data validity

into two independent concepts.

The ability to preserve shape while ignoring padding makes Masked Arrays particularly useful in scientific computing and machine learning workflows.

73 Comparing the Two Approaches

One of the most important results obtained throughout the notebook was that both approaches recover the same physical information.

For the first jet:

$$Particle\ Count = 18$$

using Boolean Masking.

$$Particle\ Count = 18$$

using Masked Arrays.

Similarly:

$$Mean\ p_T = 27.823$$

for both methods.

This agreement demonstrates that both approaches correctly identify genuine particles and successfully eliminate the influence of padding.

The distinction between the methods therefore lies not in correctness, but in how the data is represented and processed.

Boolean Masking removes unwanted data.

Masked Arrays preserve unwanted data while marking it as invalid.

Each strategy is useful under different circumstances.

74 The Importance of Handling Padding Correctly

Perhaps the most significant lesson from this exploration is the importance of handling padding correctly.

Before masking, the mean transverse momentum of the first jet was:

3.603

After masking:

27.823

The difference is dramatic.

This result clearly demonstrates that padded values can substantially distort physical observables.

In large scientific datasets, such distortions may lead to incorrect interpretations, misleading visualizations, or degraded machine learning performance.

Consequently, identifying and excluding invalid entries should be considered a fundamental preprocessing step whenever padded data is encountered.

75 Broader Relevance Beyond This Dataset

Although this notebook focused specifically on the QG jet dataset, the concepts explored are broadly applicable.

Masking techniques appear in numerous fields including:

- Particle Physics
- Scientific Computing
- Machine Learning
- Deep Learning
- Natural Language Processing
- Computer Vision
- Time Series Analysis

Examples include:

- Attention masks in Transformer models

- Missing value handling in tabular datasets
- Image segmentation masks
- Sequence padding masks
- Event selection in experimental physics

Consequently, the principles learned in this notebook extend far beyond NumPy itself. Understanding masking provides a foundation for working with many modern data processing pipelines.

76 Personal Reflection on the Exploration

This notebook was created as an exploratory study of NumPy masking techniques.

The objective was not simply to learn the syntax of a few NumPy functions, but to understand the reasoning behind masking and observe its behavior on a realistic dataset.

Beginning with a single padded jet, the exploration gradually expanded to:

- Particle-level masking
- Shape transformations
- Boolean Indexing
- Masked Arrays
- Dataset-wide analysis
- Statistical calculations
- Physics interpretation

Through this process, masking evolved from a simple programming concept into a practical data analysis tool.

The investigation also reinforced the importance of understanding how data is represented before attempting to analyze it.

Many errors in scientific computing originate not from incorrect algorithms, but from misunderstandings of the underlying data structure.

Masking provides an excellent example of this principle.

77 Final Remarks

The QG jet dataset presents a common challenge encountered throughout data science and scientific computing:

Variable-length physical information stored inside fixed-size numerical structures.

NumPy provides two powerful solutions to this challenge:

- Boolean Masking
- Masked Arrays

Boolean Masking offers a direct method for extracting physically meaningful particles.

Masked Arrays offer a flexible framework for preserving tensor structures while automatically excluding invalid values.

Both approaches are valuable, and both were shown to recover the same physically meaningful results when applied correctly.

Ultimately, this notebook demonstrates that effective data analysis is not only about performing calculations.

It is equally about understanding the structure of the data, identifying invalid information, and ensuring that computations reflect the underlying physical reality.

Masking serves as a powerful tool for achieving that goal.

78 References

The concepts, techniques, dataset descriptions, and scientific background presented throughout this report were developed using official documentation, peer-reviewed publications, and the original dataset source. The following references provide additional information regarding NumPy masking techniques, the QG jet dataset, and particle physics applications.

Dataset References

1. Komiske, Patrick T., Metodiev, Eric M., and Thaler, Jesse.
Pythia8 Quark and Gluon Jets for Energy Flow.
Zenodo Dataset Repository, Version 1.0, 2019.
Available at:
<https://zenodo.org/records/3164691>
This is the original source of the `QG_jets.npz` dataset used throughout this notebook.
2. Komiske, Patrick T., Metodiev, Eric M., and Thaler, Jesse.
Energy Flow Networks: Deep Sets for Particle Jets.
Journal of High Energy Physics (JHEP), Volume 2019, Article 121, 2019.
DOI:
[https://doi.org/10.1007/JHEP01\(2019\)121](https://doi.org/10.1007/JHEP01(2019)121)
This paper introduces the Energy Flow framework and the dataset family from which the QG jet dataset originates.
3. EnergyFlow Documentation.
EnergyFlow: Machine Learning Tools and Datasets for Particle Physics.
Available at:
<https://energyflow.network>

NumPy References

4. NumPy Developers.
NumPy Official Documentation.
Available at:
<https://numpy.org/doc/stable/>

5. NumPy Developers.
Masked Arrays (`numpy.ma`) Documentation.
Available at:
<https://numpy.org/doc/stable/reference/maskedarray.html>
6. NumPy Developers.
Indexing on ndarrays and Boolean Indexing.
Available at:
<https://numpy.org/doc/stable/user/basics.indexing.html>
7. Harris, Charles R., Millman, K. Jarrod, van der Walt, Stéfan J., et al.
Array Programming with NumPy.
Nature, Volume 585, Pages 357–362, 2020.
DOI:
<https://doi.org/10.1038/s41586-020-2649-2>

Scientific Computing References

8. VanderPlas, Jake.
Python Data Science Handbook.
O'Reilly Media, 2016.
Available online:
<https://jakevdp.github.io/PythonDataScienceHandbook/>
9. Oliphant, Travis E.
Guide to NumPy.
Second Edition.
Trelgol Publishing, USA.

Particle Physics Background

10. Particle Data Group (PDG).
Review of Particle Physics.
Available at:
<https://pdg.lbl.gov>
Referenced for the brief discussion of quarks, gluons, and jet formation presented in the dataset background section.

Figures and Visualizations

All figures, plots, histograms, heatmaps, diagrams, and visualizations included in this report were generated by the author using:

- Python
- NumPy
- NumPy Masked Arrays
- Matplotlib

using outputs obtained directly from the QG jet dataset.

Author Note

This report was created as a personal learning project to explore and understand NumPy masking techniques through practical application on a real-world particle physics dataset.

The accompanying Jupyter Notebook and source code are available in the associated GitHub repository.

End of Report

Understanding Masking in NumPy using the QG Jet Dataset: Boolean Masking vs Masked Arrays